

What is QA Testing?

Foundation of Quality Assurance

Quality Assurance (QA) Testing is the systematic process of ensuring software applications meet specified requirements and function correctly before reaching end users. As a QA professional, you act as the guardian of software quality.

Key Responsibilities

-  **Finding and documenting software defects** - Systematic bug detection and analysis
-  **Verifying features work according to specifications** - Requirements validation
-  **Ensuring applications are user-friendly** - UX/UI testing
-  **Performance and security validation** - Non-functional testing

Essential QA Skills

Technical:

Test case writing, bug reporting, automation tools, API testing

Soft Skills:

Attention to detail, analytical thinking, clear communication

Real-World Impact

Consider major companies like **Tesco** - their e-commerce platform handles millions of transactions daily. A single bug in their checkout process could cost millions in lost revenue. QA testing prevents such disasters.

Airlines Example - Croatia Airlines

An airline booking system must handle seat selection, payment processing, and passenger data accurately. One bug in the payment gateway could result in double charges or failed bookings, affecting thousands of travelers.

My Experience as QA Tester

I work as a QA Tester using tools like **TestRail** for test management, **Cypress** for automation, and **Bruno/Postman** for API testing. My daily work includes writing test cases, finding bugs, and collaborating with developers using **Jira Kanban**.

Key Achievement: Implemented automated testing that reduced manual testing time by 60% and caught critical bugs before production.

7 Testing Principles

Fundamental concepts every QA should know

1. Testing shows presence of defects

Testing can prove that defects are present, but cannot prove that there are no defects.

My Experience:

In my project, I found 15 bugs in the payment module, but I can't guarantee there are no more bugs hidden in edge cases.

2. Exhaustive testing is impossible

Testing everything is not feasible except in trivial cases. Risk analysis and priorities should guide testing efforts.

My Experience:

For our e-commerce site with 1000+ features, I prioritize testing critical paths like login, checkout, and payment over minor UI elements.

3. Early testing

Testing activities should start as early as possible in the SDLC and be focused on defined objectives.

 My Experience:

I review requirements and write test cases during the design phase, before developers start coding, to catch issues early.

4. Defect clustering

A small number of modules contain most of the defects discovered during pre-release testing.

 My Experience:

80% of bugs I find are usually in 2-3 modules like user authentication and data processing - so I focus extra testing there.

5. Pesticide paradox

If the same tests are repeated, they will no longer find new bugs. Test cases need to be reviewed and updated.

 My Experience:

I regularly update my Cypress automation scripts and add new test scenarios to catch different types of bugs.

6. Testing is context dependent

Testing is done differently in different contexts. Safety-critical software requires different testing than e-commerce sites.

 My Experience:

For payment features, I do extensive security testing, but for marketing pages, I focus more on UI/UX and browser compatibility.

7. Absence of errors fallacy

Finding and fixing defects does not help if the system built is unusable and does not fulfill user needs.

 My Experience:

Even if the app works perfectly, if users can't figure out how to complete checkout, it's still a failure - so I test usability too.

How I Apply These Principles Daily

Risk-Based Testing: I prioritize testing based on business impact and user frequency

Continuous Learning: I regularly update test cases and explore new testing techniques

Early Involvement: I participate in requirement reviews and sprint planning meetings

User-Focused: I always consider real user scenarios, not just technical requirements

SDLC - Software Development Life Cycle

Understanding development methodologies

Waterfall Model

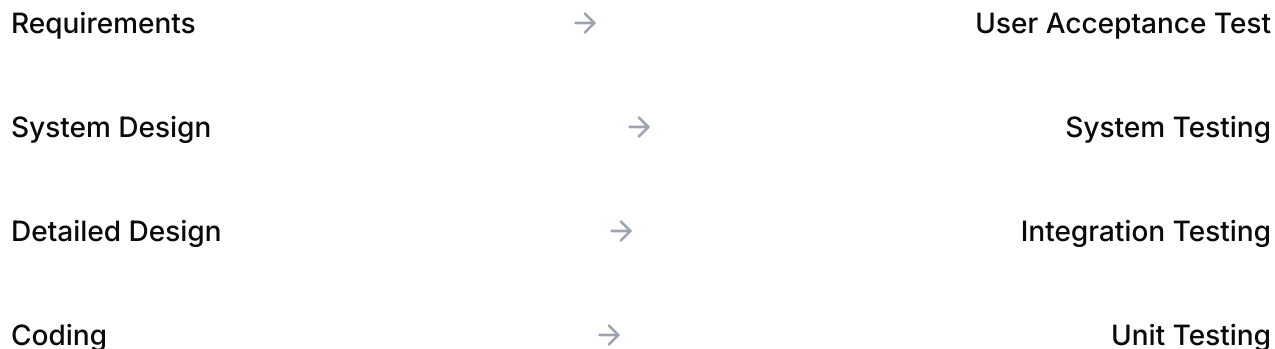
- 1 Requirements**
Gather and analyze business requirements
- 2 Design**
System architecture and UI design
- 3 Implementation**
Code development phase
- 4 Testing**
QA validation and verification
- 5 Deployment**
Release to production
- 6 Maintenance**
Ongoing support and updates

My Experience:

I've worked on projects using Waterfall where all requirements were defined upfront. Testing happened only after development was complete, which sometimes led to late bug discoveries and expensive fixes.

V-Model (Verification & Validation)

The V-Model emphasizes testing at each development phase, ensuring early defect detection.



Key Benefit: Testing strategies are planned during corresponding development phases, leading to better test coverage and early defect detection.

My Experience:

While V-Model principles are good in theory, my current project doesn't follow them. I'm not included in sprint planning or design meetings, so I can't plan tests early. I test after code is deployed to test environment - more like Waterfall approach within Agile sprints.

Agile vs Waterfall Comparison

Waterfall Testing

- Testing phase starts after development
- Detailed documentation required
- Sequential process
- Less flexibility for changes
- Good for stable requirements

Agile Testing

- Testing throughout development cycle
- Collaborative approach
- Iterative process
- High flexibility for changes
- Continuous feedback

My Experience Working in Agile

Sprint Workflow:

- **Sprint Planning:** I review user stories and acceptance criteria

My Daily Activities:

- Participate in daily standups to discuss testing progress

- **Daily Testing:** Test features as developers complete them
- **Sprint Review:** Demo tested features to stakeholders
- **Retrospectives:** Discuss testing improvements

- Collaborate with developers on testable requirements
- Create test cases for current sprint stories
- Execute regression tests for completed features
- Report bugs immediately in Jira

Key Advantage: In Agile, I can test features immediately and provide quick feedback, which leads to faster bug fixes and better quality. I also help define "Definition of Done" for each story.

STLC - Software Testing Life Cycle

Systematic approach to testing

1 Requirement Analysis

Review and understand requirements, identify testable scenarios

Activities:

- Analyze functional & non-functional requirements
- Identify test conditions
- Review acceptance criteria

Deliverables:

- Test Strategy document
- Test conditions
- Automation feasibility report

My Experience:

I participate in requirement review meetings and ask clarifying questions about acceptance criteria. I create test conditions directly in TestRail and link them to user stories in Jira.

2 Test Planning

Define test approach, scope, resources, and timeline

Activities:

- Define test scope and approach
- Estimate effort and timeline
- Identify resources and roles

Deliverables:

- Test Plan document
- Test Estimation
- Resource Planning

My Experience:

I estimate testing effort for each sprint and plan which features need manual vs automated testing. I decide whether to run Cypress tests in headless mode for faster execution or in normal mode for debugging.

3 Test Case Design & Development

Create detailed test cases and test data

Activities:

- Create test cases from requirements
- Develop automation scripts
- Prepare test data

Deliverables:

- Test Cases document
- Test Scripts
- Test Data sets

My Experience:

I write detailed test cases in TestRail with clear steps and expected results. I also create Cypress automation scripts for regression testing and prepare test data for different user scenarios.

4 Test Environment Setup

Prepare testing environment and test data

Activities:

- Setup test environment
- Install required software
- Configure test data

Deliverables:

- Environment setup document
- Test data creation
- Smoke test results

My Experience:

I coordinate with DevOps to ensure test environments are ready and run smoke tests to verify basic functionality before starting detailed testing.

5 Test Case Execution

Execute test cases and report defects

Activities:

- Execute test cases
- Log defects in bug tracking tool
- Retest fixed defects

Deliverables:

- Test execution results
- Defect reports
- Test logs

My Experience:

I execute both manual and automated tests, immediately log bugs in Jira with detailed reproduction steps and screenshots, then verify fixes during retesting.

6 Test Reporting

Analyze results and create test summary report

Activities:

- Evaluate test completion criteria
- Analyze metrics and coverage
- Prepare final report

Deliverables:

- Test summary report
- Test metrics
- Test coverage report

My Experience:

I generate test execution reports from TestRail showing pass/fail rates and present testing status in sprint reviews. I track metrics like bug density and resolution time.

7 Test Closure

Document lessons learned and archive test artifacts

Activities:

- Document lessons learned
- Archive test artifacts
- Analyze process improvements

Deliverables:

- Test closure report
- Best practices document
- Test artifacts archive

My Experience:

During sprint retrospectives, I discuss what testing approaches worked well and suggest improvements. I maintain our test automation suite and update documentation.

How I Apply STLC in My Daily Work

Agile Adaptation:

- STLC phases happen within each 2-week sprint
- Requirements analysis during sprint planning
- Test execution happens as features are developed
- Continuous reporting throughout the sprint

Tools Integration:

- TestRail for test case management and execution
- Jira for requirement traceability and bug tracking
- Cypress for automated test script development
- Bruno for API testing and validation

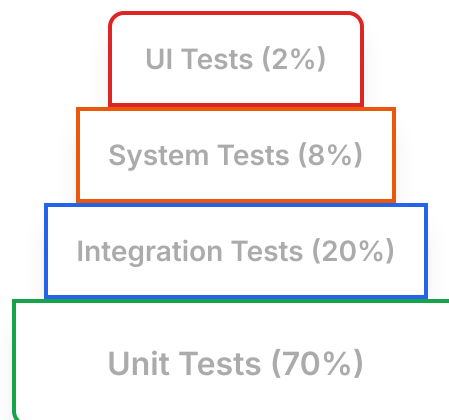
Key Success Factor: I've adapted the traditional STLC to work efficiently in Agile sprints, ensuring all phases are covered without slowing down development velocity.

Testing Levels

Different levels of software testing

Testing Pyramid

The testing pyramid shows the ideal distribution of different types of tests in a software project. More tests at the bottom (unit tests) and fewer at the top (UI tests).



⚙️ 1. Unit Testing

Testing individual components or modules in isolation. The smallest testable parts of an application.

Login Page Example:

Username Field → Unit Test

Characteristics:

- Tests individual functions/methods
- Fast execution (milliseconds)
- Easy to write and maintain
- High code coverage possible

Password Field → Unit Test

Login Button → Unit Test

- Done by developers
- Uses mocks/stubs for dependencies

My Experience:

Developers on my project handle unit testing using Java code. I focus on higher-level testing and don't review unit test coverage reports - that's handled by the development team.

Common Confusion:

Testing login form fields manually or with Cypress is NOT unit testing - that's functional/component testing. Unit testing is developers testing individual code functions (like password validation logic) directly in code, not through the UI.

2. Integration Testing

Testing the interfaces and interaction between integrated components or systems.

Integration Example:

Login Page → Database

Test: Login form communicates with user database

My Experience:

I test integrations both locally and on "test" environment. Locally, I run all services with frontend and check the database directly. On test env, when something doesn't work, I use browser Network tab to check requests and console for errors to identify integration issues.

3. System Testing

Testing the complete integrated system to verify it meets specified requirements.

System Test Areas:

- Functionality: All features work correctly
- Reliability: System stability over time
- Performance: Speed and responsiveness
- Security: Data protection and access control

My Experience:

I do end-to-end system testing like: login → create case → put case in different states → logout. This tests the complete workflow to ensure all components work together properly from start to finish.

4. User Acceptance Testing (UAT)

Final testing performed by end users to ensure the system meets business requirements.

Alpha Testing:

- Performed by internal users/employees
- Controlled environment
- Before beta testing

Beta Testing:

- Performed by external users
- Real-world environment
- Limited user group

My Experience:

UAT is handled by stakeholders and project owners on my project. They test from a business perspective to ensure features meet their requirements before release.

My Daily Testing Mix

My Daily Workflow:

- **With Jira tasks:** Manual testing, Swagger API testing, Bruno collections
- **No tasks:** Cypress automation, test maintenance, script improvements
- **Focus areas:** Functional, Integration, and System testing levels
- **Tools:** Manual testing, Cypress, Bruno, Swagger, DevTools

What I Actually Do (Higher-Level Testing):

- **Functional Testing:** Manual testing of login form validation rules
- **Component Testing:** Testing username/password fields with Cypress
- **Integration Testing:** Testing login form + backend API connection
- **End-to-End Testing:** Complete login workflow testing

My Focus: I primarily work on Integration and System testing levels, combining manual testing for new features with automation maintenance to ensure comprehensive coverage.

V-Model (Verification & Validation)

Testing throughout the development lifecycle

What is the V-Model?

The V-Model is an extension of the waterfall model where testing activities are planned in parallel with corresponding development phases. Each development phase has a corresponding testing phase.

Verification (Left Side):

- Static testing activities
- Reviews and walkthroughs
- Document analysis
- "Are we building the product right?"

Validation (Right Side):

- Dynamic testing activities
- Actual test execution
- Code execution with test data
- "Are we building the right product?"

V-Model Phase Mapping

Requirements Analysis

Gather business requirements

User Acceptance Testing

Validate user requirements



System Design

High-level architecture

System Testing

Test complete system



Detailed Design

Module-level design

Integration Testing

Test module interactions



Coding

Unit Testing

Implementation phase

Test individual modules

V-Model Benefits

✓ Advantages:

- Early test planning and design
- Better defect prevention
- Clear testing objectives
- Higher quality deliverables

✗ Disadvantages:

- Rigid and less flexible
- Difficult to accommodate changes
- No early prototypes
- High risk for complex projects

⚠ My Reality: V-Model vs Actual Practice

V-Model Theory:

- QA involved in requirements review
- Test cases planned during design
- Early defect prevention
- Parallel planning and execution

My Current Process:

- Not included in sprint planning/design
- Test after code is deployed to test env
- Sequential: Dev finishes → QA tests
- More like Waterfall within Agile sprints

🧑 My Experience:

My workflow: Task created in Jira → Developer works on it → Code merged to test environment → I test the story/task → Approve (Done) or Disapprove (back to dev). I understand V-Model benefits but haven't worked in an environment that truly implements it.

Lesson Learned: While V-Model provides excellent early defect prevention, many teams still operate in Waterfall-style sequences. I'd prefer more involvement in early phases to implement true V-Model principles.

Static vs Dynamic Testing

Two fundamental approaches to testing

👁 Static Testing

Testing without executing the code. Reviews, walkthroughs, and analysis.

Methods:

- **Code Reviews** - Peer review of source code
- **Walkthroughs** - Author explains code to team
- **Inspections** - Formal defect detection process
- **Static Analysis Tools** - Automated code analysis

Benefits:

- Early defect detection
- Cost-effective bug prevention
- Improves code quality
- Knowledge sharing

Real Example:

Reviewing login page HTML/CSS for accessibility issues, checking if proper form labels are used for screen readers.

👤 My Experience:

I don't participate in formal code reviews or use static analysis tools. My "static testing" is mainly reading user stories and discussing them in team meetings. Sometimes in daily meetings I might spot potential issues and say "hey, we need to think about this problem."

▶ Dynamic Testing



QA Testing Handbook

- **Functional Testing** - Testing features work correctly
- **Performance Testing** - Speed, load, stress testing
- **Security Testing** - Vulnerability assessment
- **Usability Testing** - User experience validation

Characteristics:

- Requires test environment
- Uses test data
- Validates actual behavior
- Can be automated

Real Example:

Actually filling out and submitting a login form with different username/password combinations to test authentication logic.

My Experience:

This is 95%+ of my work! I do manual testing, Cypress automation, API testing with Bruno, and test across multiple environments: local, test, staging, and smoke testing on production. Functional testing is what I do most.

Static vs Dynamic Comparison

Aspect	Static Testing	Dynamic Testing
Code Execution	No code execution	Code is executed
When Applied	Early development phases	After code completion
Cost	Lower cost	Higher cost
Defect Types	Logic errors, syntax issues	Runtime errors, performance issues

|| My Testing Reality: 95% Dynamic, 5% Static

My Dynamic Testing:

- **Manual Testing:** Testing user stories after development
- **Cypress Automation:** Automated regression testing
- **API Testing:** Using Bruno/Postman for backend testing

My Limited Static Testing:

- **User Story Reviews:** Reading and discussing stories with team
- **Team Discussions:** Spotting issues during daily meetings
- **No Code Reviews:** Don't participate in formal code reviews

- **Multi-Environment:** Local, test, staging, production smoke tests

- **Functional Focus:** Primarily testing feature functionality

- **No Static Tools:** Don't use automated static analysis

Example Bug Found:

Dynamic Testing Success: Found permission issues where certain user roles couldn't access features they should have. This was discovered through actual testing, not code review.

My Approach: While static testing has benefits, my role focuses heavily on dynamic testing. I validate actual functionality through hands-on testing rather than code analysis, which aligns with my current project setup.

Manual vs Automation Testing

Choosing the right approach for different scenarios

Manual Testing

Human testers manually execute test cases without automation tools.

✓ Best For:

- Exploratory testing
- Usability testing
- Ad-hoc testing
- New feature testing
- UI/UX validation

✗ Limitations:

- Time-consuming for repetitive tasks
- Human error prone
- Not suitable for load testing
- Resource intensive

Example Scenario:

Testing a new checkout flow for Tesco's website - checking if the payment process feels intuitive and secure to users.

My Experience:

70% of my daily work is manual testing. When I get Jira tasks, I immediately do manual testing. I test enhancements, new features like user editing drawers, filter pills that need to remember state across pagination pages. I follow user stories and acceptance criteria to create test cases.

Automation Testing

Using tools and scripts to execute tests automatically without human intervention.

✓ Best For:

- Regression testing
- Performance testing
- Repetitive test cases
- Data-driven testing
- Cross-browser testing

✗ Limitations:

- High initial setup cost
- Maintenance overhead
- Cannot test user experience
- Requires technical skills

Example Scenario:

Running 500 login test cases overnight to verify authentication works across different browsers and user types.

My Experience:

30% of my work is Cypress automation. I've automated login page, case page, creating cases, user page, creating users. I use intercept to check request/response payloads. Currently have 90-100 automated test cases. I automate features that are used most frequently and are really important.

Decision Matrix: When to Use What

Use Manual Testing

When:

- Testing new features for first time
- Exploring application behavior
- Checking visual elements
- Testing user workflows
- Performing accessibility testing

Use Automation

When:

- Tests need to run repeatedly
- Testing across multiple environments
- Performing load/stress testing
- Running smoke tests
- Doing regression testing

Hybrid Approach:

- Automate stable, repetitive tests
- Manual testing for new features
- Use both for comprehensive coverage
- Start manual, automate over time
- Focus automation on critical paths

🎯 My Real-World Approach: 70% Manual, 30% Automation

👤 My Manual Testing (70%):

- **New Jira tasks:** Always start with manual testing
- **Enhancement features:** User editing drawers, new UI components
- **Complex scenarios:** Filter pills remembering state across pagination
- **Follow structure:** User stories → Acceptance criteria → Test cases

🤖 My Automation (30%):

- **90-100 Cypress tests:** Login, cases, users, creating flows
- **API validation:** Intercept requests, check payloads/responses
- **Critical features:** Most frequently used, important business flows
- **When I have time:** After manual testing is complete

🕒 Example Bug Found (Manual Only):

User role change bug: When a user changes their own role, the app crashes and logs them out after 20 seconds. This type of edge case and timing issue would be very difficult to catch with automation.

Automation Maintenance Challenge:

Code changes break tests - like when we create tests for a drawer, but then project owner says "I don't want that drawer anymore" and we have to remove/update automation. UI changes require constant test maintenance.

My Strategy: Manual first for discovery and validation, then automate stable, critical paths. Sometimes I do both manual and automation for the same features for comprehensive coverage.

Functional Testing

Testing what the system does

What is Functional Testing?

Functional testing verifies that each function of the software application operates according to the requirement specification. It focuses on testing the functionality of the system.

Key Characteristics:

- Based on functional requirements
- Black box testing technique
- Validates business logic
- User-centric approach
- Input-output behavior verification

Testing Focus Areas:

- User interface functionality
- Database operations
- API functionality
- Security features
- Business workflow validation

Types of Functional Testing

Unit Testing

Testing individual components or modules in isolation.

Example - Login Function:

```
function validateLogin(username, password) {  
  if (!username || !password) return false;  
  return checkCredentials(username, password);  
}
```

Test Cases:

- Empty username → false
- Empty password → false
- Valid credentials → true
- Invalid credentials → false

My Experience:

Developers handle unit testing with Java code on my project. I focus on higher-level functional testing of complete features and user workflows.

Integration Testing

Testing interaction between integrated modules.

Big Bang Approach:

All modules integrated simultaneously and tested as a whole.

Example: Testing complete e-commerce flow:
User registration → Login → Browse products
→ Add to cart → Checkout → Payment

Incremental Approach:

Modules integrated one by one and tested at each step.

Example: First test Login + User Database, then add Product Catalog, then Shopping Cart, etc.

My Experience:

I test integration by adding new users via Swagger/Bruno API and then checking if they show up correctly in the UI and database. I focus on testing how frontend, backend, and database components work together effectively.

System Testing

Testing the complete integrated system to verify it meets specified requirements.

Real-World Example: Airlines Booking System

Flight Search

- Search by destination
- Filter by price/time
- Display available seats

Booking Process

- Seat selection
- Passenger details
- Payment processing

Confirmation

- Booking confirmation
- Email ticket
- SMS notification

My Experience:

I test complete workflows like: Login → Create case → Change status on cases → Logout. This tests the entire system end-to-end to ensure all components work together properly for real user scenarios.

User Acceptance Testing (UAT)

Final testing performed by end users to ensure the system meets business requirements.

Alpha Testing:

Beta Testing:

Internal testing by organization's employees.

Example: Tesco employees testing new online grocery ordering system before public release.

Testing by limited external users in real environment.

Example: Selected customers testing new mobile banking features before full rollout.

 My Experience:

I don't directly prepare UAT scenarios - stakeholders handle UAT. However, I participate in Jira comments where the main conversation about user stories happens, which helps inform UAT requirements.

 My Functional Testing Focus Areas

Primary Testing Areas:

1. **UI Functionality:** Testing interface elements and user interactions
2. **Business Logic:** Validating rules, calculations, and workflows
3. **End-to-End Workflows:** Complete user scenarios from start to finish

My Current Workflow:

- **Login** → User authentication
- **Create case** → Business process creation
- **Change status on cases** → Status management
- **Logout** → Session termination

Real Functional Bug Example:

Role-based access issue: When I clicked on the "Report" page with certain user roles, nothing happened - the page didn't open. This was a functional bug where the role permissions weren't properly implemented in the UI logic.

My Approach: I focus heavily on functional testing since this is where most user-facing issues occur. I test both individual features and complete business workflows to ensure everything works as expected from a user perspective.

Sample Functional Test Case

Test Case: User Registration

Test ID: TC_REG_001

Objective: Verify user can register successfully

Test Steps & Expected Results

Step 1: Navigate to registration page
Expected: Registration form displayed

Step 2: Fill valid details and submit
Expected: Success message shown

Precondition: User not registered before

Priority: High

Step 3: Check email for confirmation

Expected: Confirmation email received

Testing Techniques

White-box testing methods and coverage techniques

White Box Testing Techniques

White box testing techniques focus on the internal structure of the code. These techniques help ensure thorough testing coverage by examining different aspects of code execution.

Code Coverage Formula:

$$\text{Coverage} = (\text{Number of Executed Items} / \text{Total Number of Items}) \times 100\%$$

Example: If code has 5 lines and you test 3 lines → Coverage = $(3/5) \times 100\% = 60\%$

<> Statement Coverage

Ensures that every executable statement in the code is executed at least once during testing.

Example:

```
function validateAge(age) {  
  if (age >= 18) {  
    return "Adult";  
  }  
  return "Minor";  
}
```

Test cases: validateAge(20) and validateAge(15) achieve 100% statement coverage

My Experience:

I learned about statement coverage in QA courses, but as a functional tester, I don't use this technique directly. Developers handle code coverage analysis while I focus on testing user scenarios and business logic.

Branch Coverage

Ensures that every branch (true/false) of every decision point is executed at least once.

Example:

```
function checkAccess(age, member) {  
  if (age >= 18 && member) {  
    return "Access granted";  
  }  
  return "Access denied";  
}
```

Need tests for both true and false branches of the condition

My Experience:

While I don't measure branch coverage formally, I naturally test different conditions in my functional testing - like testing features with different user roles or permission levels to ensure all paths work correctly.

Condition Coverage

Ensures that each boolean sub-expression has been evaluated to both true and false.

Test Requirements:

- Each condition must be tested as true
- Each condition must be tested as false
- More thorough than branch coverage
- May require multiple test cases

My Experience:

🔄 Loop Testing

Focuses on testing the validity of loop constructs. Different strategies for different loop types.

Simple Loops:

- Skip the loop entirely (n=0)
- Only one pass (n=1)
- Two passes (n=2)
- m passes (n=m, typical value)
- n-1, n, n+1 passes

Nested Loops:

- Start with innermost loop
- Set outer loops to minimum
- Test innermost with simple loop strategy
- Work outward
- Continue until all tested

Concatenated Loops:

- Independent loops: test separately
- Dependent loops: test as nested
- Check loop counter dependencies
- Verify data flow between loops

Loop Testing Example:

```
function sumArray(arr) {  
  let sum = 0;  
  for (let i = 0; i <  
    arr.length; i++) {  
    sum += arr[i];  
  }  
  return sum;  
}
```

Test Cases:

- sumArray([]) - Zero iterations
- sumArray([5]) - One iteration
- sumArray([1,2]) - Two iterations
- sumArray([1,2,3,4,5]) - Multiple iterations

👤 My Experience:

I don't perform formal loop testing on code, but I do test similar scenarios functionally - like testing pagination with 0 items, 1 item, multiple items, or testing filters with empty/populated lists to ensure UI handles all cases properly.

Path Testing

Tests all possible paths through the code using cyclomatic complexity.

Cyclomatic Complexity:

$$V(G) = \text{Number of decision points} + 1$$

This determines the minimum number of test cases needed for path coverage

Example: Function with 3 decision points needs minimum 4 test cases, but thorough testing often requires more

My Experience:

I learned cyclomatic complexity in QA courses but don't calculate it in practice. However, I naturally test different user paths through the application - like different ways to create a case, various status changes, or different user roles accessing features.

Coverage Techniques Comparison

Technique	What it Covers	Strength	Weakness
Statement	Every executable statement	Easy to measure	Weakest form of coverage
Branch	Every decision outcome	Better than statement	Doesn't test all conditions
Condition	Every boolean condition	Tests individual conditions	May miss some decision outcomes
Path	Every possible execution path	Most thorough	Can be impractical for complex code

My Practical Testing Approach vs White-Box Theory

White-Box Theory (Learned in Courses):

- Statement coverage analysis
- Branch coverage calculation
- Cyclomatic complexity measurement
- Code path analysis
- Loop testing strategies

My Functional Testing Reality:

- Testing different user scenarios
- Various role and permission combinations
- Different data states (empty, populated, invalid)
- Multiple user workflows and paths
- Business logic validation

Key Insight: While I learned white-box techniques in QA courses, my daily work focuses on black-box functional testing. However, understanding these concepts helps me appreciate the testing done by developers and ensures comprehensive coverage from a user perspective.

Testing Techniques Best Practices

✓ Recommendations:

- Start with statement coverage as minimum
- Aim for 100% branch coverage
- Use condition coverage for complex logic
- Apply loop testing for all loop constructs
- Consider path testing for critical modules
- Use tools to measure coverage automatically

Coverage Goals:

- **Critical systems:** 100% branch coverage
- **Commercial software:** 80-90% coverage
- **Web applications:** 70-80% coverage
- **Prototypes:** 60-70% coverage
- Focus on quality over quantity
- Combine with black-box techniques

Performance Testing

Comprehensive performance testing strategies

👤 Load Testing

Testing system behavior under expected normal load conditions.

Typical Load Metrics:

5000 Users
Concurrent Users

< 5 sec
Response Time

1000 TPS
Transactions/Sec

👤 My Experience:

I learned load testing with JMeter during company training, running basic tests on sample pages and APIs. I tested various load scenarios but haven't yet applied this to real project environments - mainly educational practice.

↗ Stress Testing

Testing system behavior beyond normal capacity to find the breaking point.

Stress Progression:

Normal Load	5,000 users
Increased Load	10,000 users
Breaking Point	15,000+ users

🧑 My Experience:

I practiced stress testing scenarios with JMeter during training, incrementally increasing load to find breaking points. Used this for learning purposes on test pages, but not yet on production applications.

Volume Testing

Testing system performance with large amounts of data.

Volume Test Scenarios:

- Database with 10 million records
- File processing of 4GB+ files
- Memory usage with large datasets
- Network bandwidth utilization

🧑 My Experience:

I learned volume testing concepts with JMeter, testing with large datasets and multiple records during training exercises. This was part of my JMeter learning process using sample data rather than real project data.

Performance Benchmarks

Web Applications:

- Page Load: < 3 seconds
- API Response: < 200ms
- Database Query: < 100ms

Mobile Apps:

- App Launch: < 2 seconds
- Screen Transition: < 1 second
- Data Sync: < 5 seconds

Enterprise Systems:

- Transaction: < 500ms
- Report Generation: < 30 sec
- System Availability: 99.9%

🕒 My Current Performance Testing Approach

JMeter Learning Experience:

- **Training completed:** Load, stress, volume, spike testing
- **Practice environment:** Sample pages and test APIs
- **All test types covered:** Comprehensive JMeter learning
- **Challenges faced:** Old-looking UI, documentation complexity

Daily Performance Awareness:

- **Manual testing:** Notice slow-loading pages during functional testing
- **API testing:** Monitor response times in Bruno/Postman
- **Browser DevTools:** Check network timing for requests
- **User experience focus:** Report performance issues affecting users

Next Steps:

While I have solid JMeter knowledge from training, I'm ready to apply performance testing skills to real project scenarios. My current focus is functional testing, but I'm prepared to implement formal performance testing when project needs arise.

Current Status: JMeter-trained and ready to implement performance testing on actual projects. I understand the concepts and have hands-on practice, but await opportunities to apply this knowledge in production environments.

Non-Functional Testing

Testing how the system performs

What is Non-Functional Testing?

Non-functional testing evaluates the performance, usability, reliability, and other quality aspects of the software. It focuses on HOW the system performs rather than WHAT it does.

Performance Metrics:

- Response time
- Throughput
- Resource utilization
- Scalability

Quality Attributes:

- Usability
- Reliability
- Security
- Compatibility

Environmental:

- Cross-browser testing
- Mobile responsiveness
- Network conditions
- Device compatibility

Performance Testing Types

Load Testing

Testing system behavior under normal expected load conditions.

Objectives:

- Verify response time requirements
- Ensure system stability
- Validate throughput expectations
- Identify performance bottlenecks

Real Example - Tesco Online:

Normal Load: 10,000 concurrent users

Expected Response: Page load < 3 seconds

Transactions: 500 orders per minute

My Experience:

I learned load testing with JMeter during company training but haven't applied it to real projects yet. This is an area I'm ready to implement when project needs arise.

Stress Testing

Testing system behavior beyond normal capacity to find breaking point.

Airlines Example - Croatia Airlines During Holiday Rush:

Normal Load

Stress Load

Breaking Point

2,000 users booking flights
simultaneously

15,000 users during
Christmas booking rush

System fails at 20,000+
concurrent users

Goal: Ensure graceful degradation - system should slow down but not crash completely.

My Experience:

I practiced stress testing scenarios during JMeter training but haven't used this on actual projects. This is part of my performance testing skillset ready for implementation.

Volume Testing

Testing system with large amounts of data to verify performance and stability.

Database Testing Example:

Test Scenarios:

- 10 million customer records
- 100 million transaction history
- 50GB product catalog
- 1TB of user-generated content

Validation Points:

- Search response time remains < 2s
- Database queries don't timeout
- Memory usage stays within limits
- Data integrity maintained

My Experience:

I learned volume testing concepts during JMeter training but haven't applied this to production environments. This is theoretical knowledge ready for practical application.

Spike Testing

Testing system behavior under sudden, extreme load increases.

Black Friday Example - E-commerce Site:

Normal Traffic:

5,000 users



Spike Traffic (12:00 AM):

50,000 users in 2 minutes

Test Goal: Verify system can handle sudden 10x traffic increase without complete failure

My Experience:

I learned spike testing during JMeter training but haven't implemented this in real projects. This is part of my performance testing knowledge base.

Security Testing

Common Tests:

- SQL Injection attacks
- Cross-site scripting (XSS)
- Authentication bypass
- Session management
- Data encryption validation

Example: Testing login form against SQL injection: ' OR '1'='1

My Experience:

I learned security testing in courses but don't do formal security testing. My security focus is basic: checking if passwords are hidden in API responses and testing user permissions/role-based access.

Usability Testing

Evaluation Criteria:

- Ease of navigation
- User interface clarity
- Task completion time
- Error prevention
- User satisfaction

Example: Can a new user complete checkout process within 3 minutes without help?

My Experience:

Yes! I test from a user experience perspective - I think about people who will use the app. When I find usability problems, I report them to the project owner. They make the final decision about changes, but I provide the user perspective.

Compatibility Testing

Testing Matrix:

Browsers:

- Chrome 120+
- Firefox 115+
- Safari 16+
- Edge 110+

Devices:

- iPhone 12+
- Samsung Galaxy
- iPad Pro
- Desktop 1920×1080

My Experience:

I regularly test on Chrome, Edge, and Firefox. I also test responsive design using DevTools and on actual mobile phones to ensure the app works properly across different devices and screen sizes.

Reliability Testing

Metrics:

- **MTBF:** Mean Time Between Failures
- **MTTR:** Mean Time To Recovery
- **Availability:** 99.9% uptime target
- **Failure Rate:** < 0.1% transactions

My Experience:

I don't do formal reliability testing, but I naturally test system stability during long testing sessions. As hours pass while I'm doing my job, the app keeps working - that's my "unintentional" reliability testing!

My Non-Functional Testing Reality

What I Actually Do:

- **Usability Testing:** User experience perspective, report UX issues
- **Compatibility Testing:** Chrome, Edge, Firefox, responsive design
- **Basic Security:** User permissions, password visibility in responses
- **Reliability (Informal):** System works during long testing sessions

Areas I Don't Focus On:

- **Performance Testing:** JMeter knowledge but not on projects yet
- **Formal Security Testing:** Only learned in courses
- **Metrics-Based Reliability:** Don't measure MTBF/MTTR
- **Volume Testing:** Training only, not in practice

Honest Assessment: I naturally do usability and compatibility testing as part of functional testing. Performance and security testing are areas where I have theoretical knowledge but limited practical application. I focus on what users will experience rather than formal non-functional metrics.

Browser DevTools Testing

Advanced testing techniques using F12 Developer Tools

What is DevTools Testing?

Browser Developer Tools (F12) provide powerful capabilities for testing web applications beyond traditional UI testing. These tools allow QA engineers to inspect network traffic, simulate different conditions, debug issues, and validate performance in real-time.

Network Analysis:

- Request/Response inspection
- API endpoint testing
- Request blocking
- Performance monitoring

Condition Simulation:

- Offline mode testing
- Slow network speeds
- Device emulation
- Throttling CPU/Memory

Debugging:

- Console error detection
- JavaScript debugging
- Security issue identification
- Performance profiling

Network Tab Testing

Monitor and analyze all network requests to identify issues, validate API responses, and test error handling.

Request Inspection Techniques:

What to Check:

- HTTP status codes (200, 404, 500)
- Request headers and authentication
- Response time and payload size
- API endpoint URLs and parameters
- Error responses and messages

Test Scenarios:

- Login form submission validation
- File upload progress monitoring
- Search functionality API calls
- Shopping cart update requests
- Payment processing verification

Real Example - E-commerce Cart:

Test: Adding item to shopping cart

Request: POST /api/cart/add

Expected: Status 200, cart count updated

Validation: Response contains correct item ID and quantity

My Experience:

I use the Network tab daily during testing to check API requests. I've found bugs like response problems, payload issues, status code problems, and missing properties in API responses. This is my primary tool for debugging integration issues.

Request Blocking Testing

Block specific requests to test error handling and application resilience when APIs fail or are unavailable.

Common Blocking Scenarios:

What to Block:

- Authentication API endpoints
- Product data loading requests
- Image/media file requests
- Analytics and tracking scripts
- Third-party integrations

What to Validate:

- Error messages are user-friendly
- Application doesn't crash
- Graceful fallback behavior
- Retry mechanisms work
- Loading states are shown

How to Block Requests:

Step 1: Open DevTools (F12) → Network tab

Step 2: Right-click on request → "Block request URL"

Step 3: Reload page to test blocked scenario

Step 4: Validate error handling behavior

My Experience:

Yes, I use request blocking to test error handling! I block API calls to see what happens when they fail or are slow. This helps me verify that the application handles errors gracefully and shows appropriate messages to users.

Offline Mode Testing

Test Scenarios:

- Form submission when offline
- Data caching behavior
- Offline page display
- Service worker functionality
- Auto-sync when back online

How to Test: Network tab → Offline checkbox or throttle to "Offline"

My Experience:

I test offline scenarios and slow network conditions to see how the application behaves when connections are poor or unavailable. This helps identify user experience issues in real-world conditions.

Slow Network Testing

Network Presets:

- Slow 3G: 400ms latency, 400kb/s
- Fast 3G: 150ms latency, 1.6Mb/s
- Custom: Set your own speeds
- 2G conditions for worst case

Test Focus: Loading states, timeouts, progressive loading, image optimization

My Experience:

I use network throttling to test how the app performs on slow connections. This helps me identify loading issues and ensure the application provides good feedback during slow operations.

Console Debugging

Error Types to Monitor:

- JavaScript errors and exceptions
- Failed resource loading (404s)
- CORS policy violations
- Deprecated API warnings
- Security policy violations

Pro Tip: Filter by Error, Warning, Info levels to focus on critical issues

My Experience:

I check the Console sometimes for JavaScript errors during testing. Since I can see what's

Security Analysis

Security Checks:

- Sensitive data in request URLs
- Unencrypted HTTP requests
- Exposed API keys or tokens
- Missing security headers
- Cookie security settings

Warning: Never share screenshots containing sensitive authentication data

My Experience:

I check if new features expose sensitive security data in API responses or requests. I

happening locally in the terminal, I mostly focus on the Network tab, but the Console helps when I need to debug specific error scenarios.

look for things like passwords, tokens, or other sensitive information that shouldn't be visible in the Network tab.

✓ My Daily DevTools Usage vs Other Tools

DevTools (Daily Use):

- **Network tab:** Primary tool for API debugging during manual testing
- **Request blocking:** Test error handling scenarios
- **Network throttling:** Test slow/offline conditions
- **Security checks:** Look for exposed sensitive data
- **Console monitoring:** Check for JavaScript errors

Bruno/Postman (Rare Use):

- **API test creation:** Only when writing new API tests
- **Collection management:** Organizing API test suites
- **Formal API testing:** Structured API validation
- **Documentation:** API endpoint documentation

Bugs I've Found with DevTools:

- **Response problems:** Incorrect data returned from APIs
- **Payload issues:** Missing or malformed request data
- **Status code problems:** Wrong HTTP status codes (should be 200 but getting 500)
- **Missing properties:** Expected fields not present in API responses
- **Security issues:** Sensitive data exposed in network requests

My Approach: DevTools is my go-to debugging tool during manual testing. I use it daily to inspect API calls, test error scenarios, and validate responses. Bruno/Postman are mainly for formal API test creation, but DevTools is where I do most of my real-time API debugging and issue discovery.

DevTools Testing Best Practices

✓ Do's:

- Clear cache before testing to ensure fresh requests
- Document network timing for performance baselines

✗ Don'ts:

- Don't ignore console warnings and errors
- Don't test only on fast, stable connections
- Don't assume network issues are backend problems

- Test with different browser profiles and extensions disabled
- Use DevTools device emulation for mobile testing
- Save HAR files for detailed analysis
- Test API endpoints directly using console

- Don't forget to test request retries and timeouts
- Don't overlook third-party script failures
- Don't share sensitive data in bug reports

API Testing

Modern API testing strategies and tools

What is API Testing?

API (Application Programming Interface) testing is a type of software testing that involves testing APIs directly and as part of integration testing to determine if they meet expectations for functionality, reliability, performance, and security. It focuses on data exchange between different software systems.

Key Characteristics

- ✓ Tests business logic layer directly
- ✓ No user interface dependency
- ✓ Focus on data exchange validation
- ✓ Backend system integration testing

Why API Testing is Critical

- APIs are backbone of modern microservices
- 10x faster execution than UI tests
- Early detection of integration issues
- Independent of frontend changes

My API Testing Tools Experience

<> Bruno (My Choice)

My Experience:

I prefer Bruno because it's free and simple to use. I only use Bruno because it has free collections - unlimited collections vs Postman's 3-collection limit. While Postman has a better UI, Bruno meets all my API testing needs without cost limitations.

Postman (Limited Use)

My Experience:

Postman has a better UI design and more polished interface, but the free tier limitation of only 3 collections makes it impractical for my needs. I need more flexibility for different API test collections.

Why I Choose Bruno:

- Completely free with unlimited collections
- Simple and straightforward to use
- No account required
- Meets all my testing needs
- Environment variables and scripting support
- Git-friendly for version control

Why I Don't Use Postman:

- Limited to 3 collections in free tier
- Not enough for multiple projects
- Need to pay for more collections
- Bruno provides same functionality for free

My Comparison

Aspect	Bruno	Postman
Collections	Unlimited ✓	3 only ✗
UI Design	Simple	Better ✓
Cost	Free ✓	\$12+ /month
My Choice	Yes ✓	No ✗

My API Testing Workflow

⚙️ How I Actually Work with API Testing

1. Developer creates new endpoint

Rarely - only when new endpoints are created

Wait for endpoint to be ready

2. Write Bruno test collection

Really rarely - when devs write new endpoint

Create new API test in Bruno

3. Test CRUD operations

When applicable to the endpoint

Test Create, Read, Update, Delete operations

4. Validate responses

Always in every test

Check status codes, response structure, business logic

5. Run existing tests

As part of testing process

Execute test collections for regression

Key Points About My API Testing:

- **Frequency:** Really rarely write new tests - only when devs create new endpoints
- **CRUD Testing:** Yes, I test Create, Read, Update, Delete operations when applicable
- **Status Code Validation:** Always validate status codes (200, 404, 500) in script tests
- **Environment Setup:** Use Bruno environments with variables like baseUrl, API keys
- **Focus:** More time running existing tests than writing new ones

Reality Check: I don't write API tests frequently because most endpoints are created by developers who handle initial testing. I focus on comprehensive testing of new endpoints when they are developed, and maintaining existing test collections for regression testing.

HTTP Methods I Test

GET Retrieve Data

Idempotent (safe to repeat), no request body, data in URL parameters, cacheable

⚙️ Characteristics:

- ✓ Safe to repeat
- ✓ No side effects
- ✓ Can be cached
- ✓ Query parameters

👤 I Use This For:

Retrieving data and testing API responses. I validate response structure, status codes, and data integrity.

POST Create Resource

Not idempotent, request body contains data, creates new resource, not cacheable

⚙️ Characteristics:

- ✓ Creates new data
- ✓ Request body required
- ✓ Not safe to repeat
- ✓ Returns created resource

👤 I Use This For:

Creating tenants, users, and other resources. I test with valid data, invalid data, and edge cases to ensure proper validation and error handling.

PUT Update/Replace Resource

Idempotent, complete data in body, updates existing resource, replaces entire resource

Characteristics:

-  Replaces entire resource
-  Idempotent
-  Full data required
-  Updates existing

I Use This For:

Updating entire resources. I test with complete data sets and verify idempotent behavior.

PATCH Partial Update

Not idempotent, partial data in body, updates specific fields only

Characteristics:

-  Partial updates
-  Not idempotent
-  Specific fields
-  More efficient

I Use This For:

Partial updates when only specific fields need modification. More efficient than PUT.

DELETE Remove Resource

Idempotent, may have request body, removes resource, safe to repeat

Characteristics:

-  Removes data
-  Idempotent
-  Safe to repeat
-  May return deleted data

I Use This For:

Removing resources and testing proper cleanup. I verify idempotent behavior and proper error handling.

2xx Success

200 OK

Request successful

Usage: GET, PUT, PATCH responses

 **I Test This:**

Always validate 200 status in my Bruno test scripts: `expect(res.getStatus()).toEqual(200)`

201 Created

Resource created successfully

Usage: POST responses

204 No Content

Success, no content to return

Usage: DELETE responses

202 Accepted

Request accepted for processing

Usage: Async operations

4xx Client Errors

400 Bad Request

Invalid request format

Usage: Malformed JSON, missing fields

401 Unauthorized

Authentication required

Usage: Missing or invalid token

403 Forbidden

Access denied

Usage: Valid auth but no permission

404 Not Found

Resource not found

Usage: Invalid endpoint or ID

422 Unprocessable Entity

Validation errors

Usage: Valid format but business logic fails

5xx Server Errors

500 Internal Server Error

502 Bad Gateway

Server error

Usage: Unexpected server issues

Gateway error

Usage: Upstream server issues

503 Service Unavailable

Service temporarily unavailable

Usage: Maintenance or overload

504 Gateway Timeout

Gateway timeout

Usage: Upstream server timeout

API Testing Best Practices

✓ Do's - Essential Practices

- ✓ Test early and often - Include API tests in CI/CD pipeline
- ✓ Use proper assertions - Validate status codes, headers, and response body
- ✓ Test negative scenarios - Invalid inputs, missing data, edge cases
- ✓ Use environment variables - Don't hardcode URLs, API keys, or tokens
- ✓ Monitor response times - Set performance benchmarks and validate
- ✓ Validate response structure - Use schema validation for consistency
- ✓ Test authentication - Valid/invalid tokens, expired sessions
- ✓ Use dynamic test data - Generate unique data for each test run
- ✓ Document your tests - Clear descriptions and expected outcomes
- ✓ Clean up test data - Remove created test resources after testing

✗ Don'ts - Common Pitfalls

- ✗ Don't test only happy path - Include error scenarios and edge cases
- ✗ Don't hardcode test data - Use variables and generators for flexibility
- ✗ Don't ignore response headers - Validate content-type, cache headers
- ✗ Don't skip security testing - Test authorization, input validation
- ✗ Don't ignore rate limits - Respect API throttling and quotas
- ✗ Don't rely only on status codes - Validate actual response content
- ✗ Don't forget cleanup - Remove test data to avoid pollution
- ✗ Don't skip boundary testing - Test limits, large payloads, edge values
- ✗ Don't ignore error messages - Validate error response format and content
- ✗ Don't test in production - Use dedicated test environments

How I Use API Testing Tools Together

▶ My Tool Integration Strategy

Bruno (Formal API Testing):

- Create test collections for new endpoints
- Write test scripts with status code validation
- Use environment variables for different environments
- Test CRUD operations systematically
- Maintain regression test suites

DevTools (Real-time API Debugging):

- Debug API calls during manual testing
- Inspect request/response in real-time
- Check for integration issues quickly
- Validate API behavior on-the-fly
- Block requests to test error handling

My Workflow: I use DevTools for immediate debugging and issue discovery during manual testing, then create formal Bruno tests for new endpoints when developers add them. DevTools is daily use, Bruno is occasional but thorough.

Learning Resources

<> Documentation

[Bruno Documentation](#)
[REST API Tutorial](#)
[HTTP Status Codes](#)

⚡ Practice APIs

[JSONPlaceholder](#)
[ReqRes](#)
[HTTPBin](#)

🛡️ Advanced Topics

- Contract Testing (Pact)
- API Mocking & Virtualization
- GraphQL Testing
- WebSocket Testing

Pro Tip: Start with Bruno and JSONPlaceholder for practice, then gradually move to testing real APIs. Focus on understanding REST principles before diving into advanced topics like GraphQL or contract testing.

Mobile Testing

Comprehensive mobile application testing strategies

What is Mobile Testing?

Mobile testing is the process of testing mobile applications on mobile devices to ensure they function correctly, perform well, and provide excellent user experience across different devices, operating systems, and network conditions. It's one of the most challenging areas of QA due to device fragmentation and real-world usage patterns.

Key Challenges

- ⚠️ Device fragmentation (thousands of Android devices)
- ⚠️ OS version variations and update cycles
- ⚠️ Network connectivity and transition issues
- ⚠️ Touch interface interactions and gestures
- ⚠️ Battery and performance constraints

Testing Focus Areas

- ✅ Functionality across different devices
- ✅ Performance optimization and responsiveness
- ✅ Battery usage and power management
- ✅ User experience and accessibility
- ✅ Security and privacy compliance

My Mobile Testing Experience

👤 My Testing Background

- **Mobile Apps:** Tested iOS (App Store) and Android (Google Play) applications
- **Mobile Web:** Currently testing mobile web using DevTools
- **Experience:** Last project had 2 mobile apps - tested both extensively
- **Current:** No mobile app on current project, focus on web responsive

📱 Devices I Use

- **iPhone XR:** Primary iOS testing device
- **iPhone (larger model):** Different screen size testing
- **Samsung devices:** Popular Android brand testing
- **Google Pixel:** Important for Android updates!
- **Emulators:** Sometimes use, but prefer real devices

My Approach: I tested more mobile applications (native apps) than mobile web. Real devices reveal issues that emulators miss, especially UI layout problems and hardware-specific bugs.

Real Mobile Testing Issues I've Found

⚠️ iOS vs Android UI Issues

📷 Photo Upload Issues

Real Bug I Found:

On Android, the login UI looked perfect, but on iPhone the elements were positioned at the bottom of the screen. The layout completely broke on iOS even though it worked fine on Android devices.

Common iOS vs Android Issues:

- Different screen aspect ratios cause layout shifts
- Safe area handling differs between platforms
- Keyboard behavior affects input positioning
- Font rendering differences
- Status bar height variations

Real Bug I Found:

Clicked on photo upload button, selected a photo from gallery, but the photo didn't actually upload. The UI showed success but the image wasn't processed or saved.

Photo Upload Test Areas:

- Camera permission handling
- Gallery access and selection
- Image compression and resizing
- Upload progress indicators
- Network interruption during upload
- Large file size handling

My Current Mobile Web Testing Approach

DevTools Mobile Testing

What I Test:

- **Responsive design:** Different screen sizes and resolutions
- **Touch interactions:** Button sizes, tap targets
- **Mobile navigation:** Hamburger menus, mobile-specific UI
- **Form inputs:** Mobile keyboard behavior
- **Performance:** Loading times on mobile connections

My Testing Process:

- **DevTools first:** Quick responsive testing
- **Real device validation:** Verify on actual phones when needed
- **Multiple viewports:** iPhone, Android, tablet sizes
- **Network throttling:** Test on slow connections
- **Touch simulation:** Test mobile interactions

Current Reality: Since my current project doesn't have a mobile app, I focus on mobile web testing using DevTools device emulation and real device validation when necessary.

My Orientation Testing Approach

Smart Testing Strategy

My Experience:

If the app supports orientation changes, then I test both portrait and landscape modes. If rotation is disabled, I don't test orientation - I focus on testing what the app actually supports.

When I Test Orientation:

- App supports landscape mode
- Video or media viewing features
- Games with landscape support
- Camera or photo editing features
- Reading or document viewing

When I Don't Test:

- App locks to portrait only
- Forms and input-heavy screens
- Apps with disabled rotation
- Simple utility apps
- Apps designed for single orientation

Key Insight: Don't waste time testing orientation if the app doesn't support it. Focus your testing effort on features that are actually implemented and supported.

Device Testing Strategy

📱 iOS Devices (2-3 devices)

Recommended Devices:

- iPhone 14/15 (latest version)
- iPhone 12/13 (popular models)
- iPad (if tablet support needed)
- iPhone SE (small screen testing)

Advantages:

- ✓ Even 10-year-old devices can run latest OS
- ✓ More predictable update cycle
- ✓ Consistent hardware across models
- ✓ Better OS update adoption

Challenges:

- ⚠ Limited device variety
- ⚠ Expensive hardware
- ⚠ App Store review requirements

📱 Android Devices (2-3 devices)

Recommended Devices:

- Google Pixel (gets updates first!) 🔥
- Samsung Galaxy (most popular brand)
- One budget device (different performance)
- OnePlus or Xiaomi (custom ROM testing)

Advantages:

- ✓ Device fragmentation testing
- ✓ Various screen sizes and resolutions
- ✓ Different Android versions
- ✓ Cost-effective options available

Challenges:

- ⚠ Updates stop after ~2 years (reality)
- ⚠ Manufacturer customizations
- ⚠ Performance variations
- ⚠ Fragmentation complexity

🧑 Why Pixel is Important:

Google Pixel devices get Android updates first, so testing on Pixel helps catch issues with new Android versions before they reach other devices.

Critical Mobile Testing Area

Network transitions are critical for mobile apps. Wi-Fi to mobile data behaves differently than mobile to Wi-Fi. Both directions must be tested thoroughly as they can cause different issues.

Wi-Fi to Mobile Data

Test transition from Wi-Fi to cellular network

Test Steps:

1. Start using app on Wi-Fi (upload, download, form filling)
2. Turn off Wi-Fi while operation is in progress
3. App should automatically switch to mobile data
4. Continue the operation seamlessly
5. Verify no data loss or corruption

Expected: Seamless transition, no interruption to user experience

Common Issues: Connection timeout, Data loss, Failed uploads, Session drop

Mobile Data to Wi-Fi

Test transition from cellular to Wi-Fi network

Test Steps:

1. Use app on mobile data (streaming, browsing)
2. Enter Wi-Fi range and connect to network
3. App should detect better connection
4. Automatically switch to Wi-Fi
5. Optimize bandwidth usage accordingly

Expected: Automatic switch to better connection, improved performance

Common Issues: Duplicate downloads, Connection conflicts, Speed issues

No Network Connection

Test offline functionality and error handling

Test Steps:

1. Turn off all network connections (airplane mode)
2. Try to use app features
3. Test cached content availability
4. Attempt network operations
5. Verify proper error messages

Expected: Graceful degradation, clear error messages, offline functionality

Common Issues: App crashes, Poor error messages, No offline support

Poor Network Conditions

Test app behavior on slow/unstable connections

Test Steps:

1. Simulate 2G/3G network conditions
2. Test app loading and responsiveness
3. Try uploading large files or images
4. Test timeout handling
5. Verify retry mechanisms

Expected: Proper loading states, retry mechanisms, timeout handling

Common Issues: Long loading times, No progress indicators, Failed operations

📁 Background/Foreground Testing

🔥 Critical Testing Area

Test how the app behaves when it goes to background - put in background and turn off screen, or just turn off screen, or simply open a new app. Each scenario behaves differently!

Background + Screen Off Critical

Most comprehensive background test scenario

Test Steps:

1. Open app and perform an action (like filling a form)
2. Press home button to put app in background
3. Turn off screen using power button
4. Wait 5+ minutes (simulate real-world pause)
5. Turn screen back on and return to app

Expected: App should resume exactly where user left off, maintaining all form data and application state

Common Issues: Form data lost, App restarts, Session timeout, Memory cleared

Screen Off Only High

Tests screen timeout behavior without backgrounding

Test Steps:

1. Use app actively (scrolling, typing, etc.)
2. Press power button to turn off screen only
3. Wait 2-3 minutes
4. Turn screen back on (app still in foreground)

Expected: App should maintain exact state, no data loss, immediate responsiveness

Common Issues: Screen flicker, Layout shifts, Input focus lost

App Switching High

Tests multitasking and app switching behavior

Test Steps:

1. Open your app and navigate to important screen
2. Open another app (camera, messages, phone call)
3. Use the other app for 1-2 minutes
4. Return to your app via task switcher

Expected: No data loss, proper state recovery, smooth transition back

Common Issues: App restart, Data reset, Navigation stack lost

Incoming Call Interruption Critical

Tests app behavior during phone calls (critical scenario)

Test Steps:

1. Use app actively (especially during important actions)
2. Receive or make a phone call
3. Handle the call (accept/decline/talk)
4. Return to app after call ends

Expected: App gracefully handles interruption, preserves user progress

Common Issues: Transaction lost, Form data cleared, Session expired

My Mobile Testing Workflow

⚙️ How I Approach Mobile Testing

Device Setup:

- iPhone XR for iOS testing
- Samsung device for Android
- Google Pixel (important!)
- Emulators as backup
- Keep devices charged

Testing Focus:

- UI layout differences
- Photo upload functionality
- Orientation support check
- Different screen sizes
- Cross-platform consistency

Current State:

- No mobile app currently
- Focus on mobile web
- DevTools for responsive
- Real device validation
- Previous project: 2 apps

My Experience: I've tested more native mobile applications than mobile web. Real devices reveal critical issues that emulators miss, especially UI layout problems and hardware functionality.

Screen Size & Resolution Testing

📱 Multiple Device Testing

👤 My Approach:

I test different screen sizes and resolutions because the same app can look completely different on various devices. What looks perfect on iPhone XR might break on a smaller Android device.

Screen Sizes I Test:

- **Small phones:** iPhone SE, smaller Android
- **Standard phones:** iPhone XR, Galaxy S series
- **Large phones:** iPhone Plus/Max, Note series
- **Tablets:** iPad, Android tablets (if supported)
- **Different ratios:** 16:9, 18:9, 19.5:9

What I Look For:

- **Layout breaking:** Elements overlapping or misaligned
- **Text cut-off:** Labels or content not fully visible
- **Button sizes:** Too small to tap or too large
- **Image scaling:** Distorted or improperly sized images
- **Navigation issues:** Menus not working properly

Real Devices vs Emulators

📱 Real Devices (My Preference)

👤 My Experience:

I prefer real devices because they reveal issues emulators miss - like hardware-specific problems, actual touch interactions, and performance issues under real conditions.

- ✓ **Real hardware behavior:** Actual performance and limitations
- ✓ **Touch interactions:** Real finger gestures and pressure
- ✓ **Camera/sensors:** Actual hardware functionality
- ✓ **Network conditions:** Real cellular and Wi-Fi
- ✓ **Battery impact:** Actual power consumption

💻 Emulators (Sometimes Use)

👤 When I Use Emulators:

I sometimes use emulators for quick testing or when I need specific OS versions, but always validate critical issues on real devices.

✓ Good For:

- Quick UI layout checks
- Different OS version testing
- Basic functionality validation
- Development environment testing

✗ Limitations:

- Miss hardware-specific issues
- Performance not realistic
- Can't test camera/sensors properly
- Network simulation limited

OS Version Impact Testing

⚠️ OS Update Warning from Industry Experience

"If you're working on a native app, there's a high chance that when a new OS is released, everything will break and you'll need full regression testing. React Native apps have lower risk but still need smoke testing. Always monitor new OS releases closely!"

Native Applications (High Risk)

- ⚠️ **High Risk:** New OS can break core functionality
- ⚠️ **Required:** Full regression testing needed
- ⚠️ **Timeline:** Test immediately after OS beta release
- ⚠️ **Focus:** Core functionality, UI elements, permissions

React Native/Flutter (Lower Risk)

- ✓ **Lower Risk:** Framework handles most OS differences
- ✓ **Required:** Smoke testing on new OS versions
- ✓ **Focus:** Happy path scenarios and critical features
- ✓ **Benefit:** Framework updates handle compatibility

Right-to-Left (RTL) Languages

Expert Warning:

Arabic text flows right-to-left, which means many screens can completely break. Just switching the language can cause the entire UI to fall apart - test this for every supported RTL language!

Languages:

- Arabic
- Hebrew
- Persian
- Urdu

Testing Approach:

- ✓ Test every screen with RTL language
- ✓ Check text alignment and overflow
- ✓ Verify UI element positioning
- ✓ Test navigation and user flows
- ✓ Validate form layouts and inputs

Text Expansion

Languages:

- German
- French
- Spanish
- Dutch

Testing Approach:

- ✓ Test UI with longest text variations
- ✓ Verify button and field sizing
- ✓ Check text truncation handling
- ✓ Validate responsive layouts

Number and Date Formats

Variations:

- US: 1,234.56 vs EU: 1.234,56
- Date formats: MM/DD/YYYY vs DD/MM/YYYY
- Currency positioning: \$123 vs 123\$
- Time formats: 12h vs 24h

Testing Approach:

- ✓ Test all numeric inputs
- ✓ Verify date picker behavior
- ✓ Check currency display
- ✓ Validate calculation accuracy

Real-World Mobile Testing Scenarios

E-commerce App

Shopping Cart Network Transition

Context: Real scenario that happens to users daily

Test Steps:

1. Add items to cart while on home Wi-Fi
2. Leave house (switch to mobile data automatically)
3. Continue shopping, add more items
4. Enter store with Wi-Fi (auto-switch back)
5. Proceed to checkout and payment

Expected: All cart items preserved, no duplicates, seamless checkout

Common Failures: Cart cleared, Duplicate items, Payment failures

Banking App

Biometric Authentication After OS Update

Context: OS updates often break biometric authentication

Test Steps:

1. Set up fingerprint/face ID login
2. Update iOS/Android to new version
3. Restart device completely
4. Open banking app
5. Try biometric authentication

Expected: Biometric auth continues working without re-setup

Common Failures: Requires re-setup, Falls back to password, App crashes

Social Media App

Photo Upload on Poor Network

Context: Users often have poor network while taking photos

Test Steps:

1. Take high-quality photo (large file size)
2. Start upload on Wi-Fi
3. Move to area with poor 2G/3G signal
4. Monitor upload progress and behavior
5. Switch back to Wi-Fi or better signal

Expected: Upload resumes properly, no corruption, progress maintained

Common Failures: Upload fails, Corrupted images, No retry mechanism

Navigation App

GPS During Phone Call

Context: Critical scenario for safety

Test Steps:

1. Start navigation for important trip
2. Receive incoming phone call
3. Answer call and talk while driving
4. Continue following navigation
5. End call and continue trip

Expected: Navigation continues uninterrupted, voice guidance works

Common Failures: Navigation stops, Audio conflicts, Route lost

Expert Insights from Industry Experience

Payment Integration Testing

Loading States and Interruptions

"Test edge cases: blocked card, insufficient funds, stolen card, expired card, wrong CVV. Integration with payment systems requires thorough testing of all failure scenarios."

Practical Advice:

- Test with different card types (Visa, MasterCard, Amex)
- Simulate network timeouts during payment
- Test partial payment failures
- Verify refund and chargeback handling
- Test payment in different currencies

Impact: Payment failures can cause immediate revenue loss and customer frustration

"Test what happens when user exits app during loading, then returns. Does it continue loading or restart? This is a common real-world scenario."

Practical Advice:

- Test loading interruption at different stages
- Verify proper loading state recovery
- Test with slow network conditions
- Check memory management during loading
- Validate progress indicators accuracy

Impact: Poor loading experience leads to app abandonment

Device Management Strategy

"Keep devices charged, don't update all iOS devices to same version, coordinate device usage in team. One device will always be problematic!"

Practical Advice:

- Maintain device rotation schedule
- Keep at least one device on previous OS version
- Document device-specific issues
- Share device usage calendar with team
- Have backup devices ready

Impact: Poor device management slows down testing and misses critical bugs

Version Display Requirement

"App version must be displayed somewhere (usually in Settings). Essential for bug reports - you need to know which build the client is using on production."

Practical Advice:

- Show version in Settings or About screen
- Include build number for internal tracking
- Make version easily accessible to users
- Consider showing version in crash reports
- Update version display for each release

Impact: Without version info, reproducing production bugs becomes nearly impossible

Network Transition Reality

"When switching from Wi-Fi to mobile data, the behavior is different than switching from mobile to Wi-Fi. This isn't the same behavior pattern - you must test both directions thoroughly."

Practical Advice:

- Test both transition directions separately
- Monitor data usage during transitions
- Check for duplicate network requests
- Verify proper timeout handling

OS Update Impact

"If you're working on a native app, there's a high chance that when a new OS is released, everything will break and you'll need full regression testing. React Native apps have lower risk but still need smoke testing."

Practical Advice:

- Monitor OS beta releases actively
- Set up test devices with beta OS
- Plan regression testing cycles

- Test with different network speeds

Impact: Network transition failures cause data loss and poor user experience

- Test immediately after OS updates
- Keep documentation of OS-related issues

Impact: OS updates can break apps for millions of users overnight

Comprehensive Mobile Testing Checklist

Pre-Testing Setup

- ✓ All devices charged and ready (>80% battery)
- ✓ App installed on all target devices
- ✓ Test data and scenarios prepared
- ✓ Test user accounts created for different roles
- ✓ Network configurations tested (Wi-Fi, 4G, 5G)
- ✓ Device usage schedule coordinated with team

Core Testing Areas

- ✓ Network switching (Wi-Fi ↔ Mobile data) both directions
- ✓ Orientation changes (if supported)
- ✓ Performance metrics (loading, transitions)
- ✓ Memory management validation
- ✓ Background/foreground behavior (all scenarios)
- ✓ RTL language testing (Arabic, Hebrew)
- ✓ Battery usage monitoring

Device-Specific Testing

- ✓ Screen size variations (small, large, tablet)
- ✓ Hardware feature testing (camera, GPS, sensors)
- ✓ Manufacturer customizations (Samsung, Xiaomi)
- ✓ OS version differences (current, previous)
- ✓ Performance variations (high-end vs budget)
- ✓ Storage constraints (low storage scenarios)

⚡ Real-World Scenarios

- ✔ Interruption handling (calls, notifications)
- ✔ Poor network conditions (2G, unstable connection)
- ✔ Low battery behavior and power saving modes
- ✔ Multitasking and app switching
- ✔ Long usage sessions (memory leaks)
- ✔ Edge cases and error conditions

🕒 Post-Testing

- ✔ Bug reports created with device details
- ✔ Device-specific issues documented
- ✔ Performance data collected and analyzed
- ✔ Test results shared with development team
- ✔ Devices cleaned and prepared for next cycle
- ✔ Lessons learned documented for future testing

🛡️ App Store Compliance - Critical Requirements

Apple App Store

Account Deletion (CRITICAL)

CRITICAL

Must provide easily accessible account deletion option

🍏 Expert Warning:

If you create an account, you **MUST** have an option to delete the account or Apple will reject your app. This must be easily accessible and visible to users - Apple's test team will specifically look for this!

Implementation Requirements:

- ⚠ Add 'Delete Account' option in Settings
- ⚠ Make it easily discoverable
- ⚠ Provide clear confirmation flow
- ⚠ Actually delete user data (not just deactivate)

Consequence: App will be rejected if not implemented

Target API Level

Must target latest Android API level (within 1 year)

Consequence: App updates will be rejected

Data Safety Section

Must declare data collection and sharing practices

Consequence: Required for all app submissions

Mobile Testing Best Practices

✔ Do's - Essential Practices

- ✔ Test on real devices - Emulators miss critical hardware-specific issues
- ✔ Rotate device focus weekly - Each device will reveal unique problems
- ✔ Test network transitions - Critical for modern mobile apps
- ✔ Monitor app in background - Test all state preservation scenarios
- ✔ Show app version prominently - Essential for production bug reports
- ✔ Test with low storage - Apps behave differently when storage is limited
- ✔ Use different user accounts - Fresh vs returning user experiences vary
- ✔ Test app updates - Ensure smooth upgrade experiences
- ✔ Document device quirks - Track device-specific issues for future reference

✘ Don'ts - Common Pitfalls

- ✘ Don't rely only on emulators - Miss real-world hardware interactions
- ✘ Don't test on single device - Device fragmentation is very real
- ✘ Don't ignore orientation - Test both portrait and landscape if supported
- ✘ Don't skip OS updates - New OS versions can break existing functionality
- ✘ Don't forget permissions - Test all permission scenarios thoroughly
- ✘ Don't overlook localization - RTL languages can completely break UI
- ✘ Don't ignore app store rules - Compliance failures cause rejection
- ✘ Don't skip edge cases - Real users encounter unexpected scenarios
- ✘ Don't forget cleanup - Remove test data to avoid interference

✓ Test during peak usage - Real-world conditions matter

✗ Don't test only latest devices - Support older devices users actually use

Mobile Testing Reality Check

What I've Learned

- Different devices reveal different issues
- iOS and Android can have completely different layouts
- Photo upload is often problematic on mobile
- Real devices catch issues emulators miss
- Google Pixel is important for Android updates

Practical Tips

- Test orientation only if app supports it
- Focus on screen sizes your users actually use
- Always test photo/camera functionality thoroughly
- Don't rely only on emulators for final testing
- Document device-specific issues for future reference

Key Takeaway: Mobile testing requires real devices to find real issues. My experience shows that the same app can behave completely differently on different devices, especially between iOS and Android platforms. Focus your testing on what users actually use.

Black Box vs White Box Testing

Testing approaches based on code knowledge

Black Box Testing

Testing without knowledge of internal code structure. Focus on inputs and outputs.

Techniques:

- **Equivalence Partitioning** - Group similar inputs
- **Boundary Value Analysis** - Test edge values
- **Decision Table Testing** - Test business rules

- **State Transition Testing** - Test state changes

Example - Login Form:

Input	Expected	Result
Valid user/pass	Login success	✓ Pass
Invalid user	Login failed	✓ Pass
Empty fields	Validation error	✓ Pass

Real-World Use:

Testing Croatia Airlines booking system by trying different passenger counts, dates, and destinations without knowing the backend database structure.

My Experience:

This is mainly what I do! I get a Jira task → read the requirements → test the functionality on the web app. I don't think about formal black box techniques, I just test what the feature should do without looking at code.

<> White Box Testing

Testing with full knowledge of internal code structure, logic, and design.

Techniques:

- **Statement Coverage** - Execute every code line
- **Branch Coverage** - Test all if/else paths
- **Path Coverage** - Test all possible paths
- **Condition Coverage** - Test all conditions

Code Example:

```
if (user.isValid() && user.isActive()) {  
    loginUser(user);  
} else {  
    showError("Invalid credentials");  
}
```

Test cases: valid+active user, valid+inactive user, invalid user

Real-World Use:

Unit testing payment processing code to ensure all branches (successful payment, insufficient funds, network timeout) are properly tested.

My Experience:

I only use white box testing for Cypress automation tests. I need to look at the code to add global attributes or understand the DOM structure for my test selectors. Otherwise, developers handle most white box testing.

Gray Box Testing (Hybrid Approach)

Combination of black box and white box testing - limited knowledge of internal workings.

Characteristics:

- Partial code knowledge
- Access to design documents
- Integration testing focus
- API testing

Best For:

- Integration testing
- Penetration testing
- Matrix testing
- Regression testing

Example:

Testing API endpoints for e-commerce cart - knowing the API structure but not the internal database queries.

My Experience:

This describes my API testing perfectly! When I test APIs in Bruno/Postman or Swagger, I know the API structure and endpoints, but I don't know the internal backend code. I understand what the API should do but not how it's implemented.

My Real Testing Approach

Black Box (Most Used):

- **Daily workflow:** Jira task → read → test on web app
- **No code knowledge:** Just test functionality
- **Focus:** Does the feature work as expected?
- **Approach:** User perspective testing

Gray Box (API Testing):

- **Bruno/Postman:** Know API structure
- **Swagger testing:** Understand endpoints
- **Limited knowledge:** API specs but not backend code
- **Integration focus:** How APIs connect

White Box (Cypress Only):

- **Automation tests:** Need to see DOM structure
- **Global attributes:** Add test selectors to code
- **Limited use:** Only for Cypress automation
- **Purpose:** Create reliable test selectors

My Reality: I mainly do black box testing without thinking about formal techniques. When I get a Jira task, I just test the feature like a user would. I only need code knowledge for Cypress

tests or when testing APIs where I know the structure but not the implementation.

When I Use Each Approach

Black Box Testing Examples:

- Testing new form functionality from Jira task
- Checking if login page works correctly
- Validating user permissions and role access
- Testing business workflows without seeing code

Gray Box Testing Examples:

- Testing user creation API in Bruno with known endpoints
- Validating API responses without knowing backend logic
- Integration testing between frontend and API
- Swagger testing with API documentation

White Box Testing Examples:

- Adding data-cy attributes to elements for Cypress tests
- Understanding DOM structure for automation selectors
- Looking at code to write better Cypress tests
- Global configuration changes for test environment

Key Insight: I don't think about these categories when testing - I just use whatever knowledge I have available. Most of my testing is naturally black box because I test features like users would use them.

Bug Reporting

Effective defect identification and documentation

What is a Bug?

A bug (or defect) is a flaw in a software system that causes it to behave in an unintended or unexpected way. It represents a deviation from the expected functionality as defined in the requirements.

Types of Bugs:

- Functional bugs
- Performance issues
- UI/UX problems
- Security vulnerabilities
- Compatibility issues
- Data corruption

Common Causes:

- Coding errors
- Requirement misunderstanding
- Design flaws
- Integration issues
- Environmental factors
- Human mistakes

Impact Areas:

- User experience
- System performance
- Data integrity
- Business operations
- Security risks
- Financial losses

My Bug Reporting Workflow

My Tools & Process

- **Primary tool:** Jira for all bug reports
- **Alternative:** Teams planner or Teams channel (if agreed by team)
- **Key principle:** Consistency - bugs in ONE place, not 2-3-4 places
- **Evidence tool:** ScreenRec app for recording videos
- **Process:** Find bug → Go to Jira → Create bug report with my structure

Communication & Decision Making

- **Priority/Severity:** Sometimes project manager/team lead helps, sometimes I decide
- **Assignment:** I assign developers to tickets
- **Discussion:** We talk about bugs on daily standup or calls
- **Unclear bugs:** Really rarely from my side, but I fix if needed
- **Challenge:** Sometimes get stories with no acceptance criteria or just title

My Philosophy: Consistency is important - have bug reports in one place, not scattered across multiple tools. Always follow the same template and structure for clear communication.

My Bug Report Template

My Experience:

I follow a specific template and always use it. This is my consistent structure for every bug report in Jira.

My Jira Bug Report Template

Structure I use for every bug report

Summary

Login button not responding on mobile
Safari

Version Used

v2.1.3

Frequency

Always / Sometimes / Rarely

OS

iOS 16.0

Steps to Reproduce

1. Open application on iOS device
2. Navigate to login page
3. Enter valid credentials
4. Tap login button

Browser

Safari

Expected Result

User should be logged in and redirected
to dashboard

Env

Testing environment

Actual Results

Login button does not respond to touch
events

Screen Resolution

1170 × 2532

Device

iPhone 14 Pro

Workaround (sometimes using)

Use desktop version or refresh page
twice

Notes (sometimes using)

Issue occurs only on iOS Safari, works
fine on Chrome mobile

Evidence

  Always include screenshots,  Sometimes ScreenRec video recordings

Bug Lifecycle

Bug Status Flow

1

New

Bug discovered and reported

2

Assigned

Assigned to developer for fixing

3

Fixed

Developer has resolved the issue

4

Closed

Bug verified as fixed and closed

Priority vs Severity

Severity

Impact on system functionality - How much the bug affects the system's operation.

Critical System crash, data loss

Major Feature not working

Medium UI issues, typos

Low Cosmetic issues

Priority

Urgency of fixing - How quickly the bug needs to be resolved based on business needs.

High Fix immediately

Medium Fix in current release

Low Fix in future release

Example Scenarios:

High Priority + Critical Severity: Payment system crashes during checkout

High Priority + Low Severity: Company logo missing on homepage before product launch

Low Priority + Critical Severity: Admin panel crashes (affects few users)

Low Priority + Low Severity: Text alignment issue in footer

My Priority & Severity Examples

My Experience:

Sometimes project manager or team lead helps me decide priority and severity, but sometimes I decide myself based on user impact.

High Priority Examples

Low Priority Examples

User cannot login

Why high priority: Blocks core functionality, affects all users

Payment processing fails

Why high priority: Direct business impact, revenue loss

Login page shows "Username" instead of "userName"

Why low priority: User can still login, only text is wrong

Button text alignment slightly off

Why low priority: Cosmetic issue, doesn't affect functionality

Real-World Testing Challenges

⚠ Challenges I Face as QA

👤 My Real Experience:

Sometimes as QA you get really bad stories with no acceptance criteria, or sometimes without story and you only have title. Then you need to ask devs, team lead, or project manager what they mean with that story.

Common Issues I Encounter:

- Stories with no acceptance criteria
- Tasks with only a title, no description
- Unclear requirements from stakeholders
- Missing edge case scenarios
- Ambiguous business rules

How I Handle These Situations:

- Ask developers for clarification
- Reach out to team lead or project manager
- Request proper acceptance criteria
- Clarify scope and expected behavior
- Document assumptions and get approval

Key Learning: As QA, you often need to be proactive in getting the information you need to test properly. Don't assume - always ask for clarification when requirements are unclear.

Bug Reporting Best Practices

✅ ✓ Do's:

- Write clear, descriptive titles
- Provide detailed steps to reproduce
- Include screenshots/videos
- Specify environment details
- Set appropriate priority and severity

✗ Don'ts:

- Don't use vague descriptions
- Don't report duplicate bugs
- Don't skip reproduction steps
- Don't assume everyone knows the context

- Test on multiple environments
- Verify bug before reporting
- Use consistent template structure
- Keep all bugs in one place (like Jira)

- Don't report multiple issues in one bug
- Don't forget to include evidence
- Don't set wrong priority/severity
- Don't scatter bugs across multiple tools
- Don't test without proper requirements

My Approach: Always use the same template structure, keep bugs in one place for consistency, include screenshots/videos, and ask for clarification when requirements are unclear. Clear communication is key!

Test Case Writing

Creating effective test cases

My Test Case Writing Approach

My Tools & Process

- **Tool:** TestRail for writing and organizing test cases
- **Template:** Simple approach - only titles starting with "Verify that..."
- **When I write:** When I have time before testing, or later if "fast" testing needed
- **Detail level:** Only titles, no detailed steps (works for my projects)
- **Focus:** Making sure the task/story works

My Workflow

- **Preparation:** Look at acceptance criteria
- **Writing:** Add test cases based on criteria + my edge cases
- **Execution:** Run tests in TestRail
- **Reporting:** Generate reports when finished
- **Reuse:** Use for regression and smoke testing

My Philosophy: Important for test cases that you have user story with acceptance criteria. If you don't have that info, then you are in trouble - you don't know what you need to test!

My Real TestRail Examples

My Experience:

I write test cases in TestRail with simple titles. I don't write detailed test cases because I didn't need them on my last two projects - only needed to verify the task/story works.

Test Examples:

ID	Test Case Title (My Format)
----	-----------------------------

C385	Verify that user can create a new project with all required fields
------	--

C386	Verify that validation errors are shown for empty required fields
------	---

C387	Verify that user receives confirmation email after project creation
------	---

C388	Verify that user cannot click "Create" button when required fields are empty
------	--

C389	Verify that user get toast message after project creation
------	---

My Template: All test cases start with "Verify that..." and focus on what functionality should work. Simple titles that clearly describe what I'm testing without detailed steps.

When I Use Test Cases vs Exploratory Testing

When I Write Test Cases:

- **Important features:** When it's a critical functionality
- **When I have time:** Before testing if schedule allows
- **Regression testing:** For smoke tests and regression
- **Complex features:** When I need to track coverage
- **With good criteria:** When user story has clear acceptance criteria

When I Do Exploratory:

- **"Fast" testing:** When quick testing is needed
- **Simple features:** When functionality is straightforward
- **Time pressure:** When deadlines are tight
- **Unclear requirements:** When acceptance criteria is missing
- **Bug investigation:** When exploring issues

My Approach: Sometimes exploratory testing, sometimes test cases. If it's an important feature, that's a good time to write test cases. The key is having good user stories with acceptance criteria!

My Test Case Creation Process

🕒 Step-by-Step Process:

1

Read User Story

Look at acceptance criteria

2

Write Test Cases

Based on criteria + edge cases

3

Run in TestRail

Execute and mark results

4

Generate Report

When testing finished

What I Include in Test Cases:

From Acceptance Criteria:

- Core functionality tests
- Business rule validations
- User workflow tests
- Expected behavior verification

My Additional Edge Cases:

- Error scenarios
- Boundary value testing
- Negative test cases
- Integration points

Test Case Structure

Essential Components:

- ✓ **Test Case ID:** Unique identifier (TC_001)
- ✓ **Test Case Title:** Clear, descriptive name
- ✓ **Objective:** What you're testing
- ✓ **Preconditions:** Setup requirements
- ✓ **Test Steps:** Detailed actions
- ✓ **Expected Results:** What should happen
- ✓ **Postconditions:** Cleanup steps

Test Case Attributes:

Priority: High/Medium/Low

Test Type: Functional/Non-functional

Test Level: Unit/Integration/System

Test Data: Required input data

Environment: Test environment details

Author: Test case creator

Creation Date: When created

Execution Status: Pass/Fail/Blocked

My Approach:

I keep it simple - my TestRail test cases are just titles starting with "Verify that..." I don't need detailed steps because I know what to test from the acceptance criteria and my experience.

Sample Detailed Test Case: User Login

This is an example of a detailed test case structure (for reference, though I use simpler titles)

Test Case ID: TC_LOGIN_001

Title: Verify that user can successfully login with valid credentials

Objective: Test login functionality with correct username and password

Priority: High

Precondition: User has valid account, browser is open

Test Data: Username: testuser@example.com
Password: Test123!

Test Steps & Expected Results:

Step 1: Navigate to login page (www.example.com/login)
Expected: Login form is displayed with username and password fields

Step 2: Enter valid username in username field
Expected: Username is entered successfully

Step 3: Enter valid password in password field
Expected: Password is masked and entered successfully

Step 4: Click Login button
Expected: User is redirected to dashboard page

Step 5: Verify user is logged in
Expected: User profile/logout option is visible

Test Execution Results

 PASSED

 FAILED

 BLOCKED

Definition: Test executed successfully and met all expected results

Action: Mark as passed, move to next test case

Documentation: Record execution date and tester name

Definition: Test did not meet expected results, defect found

Action: Create bug report, assign to development team

Documentation: Record failure details and attach evidence

Definition: Test cannot be executed due to external factors

Action: Identify and resolve blocking issue

Documentation: Record reason for blocking and resolution steps

Test Case Writing Best Practices

✓ Do's:

- Write clear, concise test steps
- Use simple language
- Include specific test data
- Make test cases independent
- Cover both positive and negative scenarios
- Review and update regularly
- Start titles with "Verify that..." for clarity
- Base test cases on acceptance criteria
- Add your own edge cases

✗ Don'ts:

- Don't write vague or ambiguous steps
- Don't assume prior knowledge
- Don't create dependent test cases
- Don't skip expected results
- Don't use complex technical jargon
- Don't forget to specify test data
- Don't write test cases without acceptance criteria
- Don't over-complicate when simple titles work
- Don't ignore edge cases and error scenarios

My Key Learning: The most important thing for test cases is having a user story with acceptance criteria. Without that, you're in trouble because you don't know what you need to test! Keep it simple but effective.

Regression Testing

Ensuring new changes don't break existing functionality

What is Regression Testing?

Regression testing is the process of testing existing software functionality to ensure that new code changes, bug fixes, or new features haven't negatively impacted the existing working features.

Key Objectives:

- Verify existing functionality still works
- Ensure new changes don't introduce bugs
- Maintain software quality and stability
- Validate system integration after changes
- Confirm bug fixes don't create new issues

When to Perform:

- After bug fixes
- After new feature implementation
- After code refactoring
- Before major releases
- After environment changes

My Regression Testing Approach

My Experience:

I focus on practical, targeted regression testing. I don't test everything - I test what's related to the changes and what's important. If I tested everything all the time, I would need a lot of time for testing one feature!

After Bug Fixes

- **When dev fixes bug:** I check that the bug is actually fixed
- **Test related functionality:** Check features around the bug fix
- **Manual approach:** I have Cypress tests but check manually for now
- **Verify the fix:** Make sure the original issue doesn't happen anymore

After New Features

- **Focused testing:** Only test the area of the new feature
- **Smart approach:** If new thing in user details, test user details area
- **Don't test everything:** Won't test all user management for small changes
- **Time efficient:** Avoid unnecessary testing to save time

My Real Regression Testing Examples

Example 1: Password Requirement Changes

My Real Example:

"If password needs new requirement, I will test if that is fixed or created and test login etc..."

What I Test:

- Password creation with new requirements
- Login with old passwords (should still work)
- Password validation messages
- Password reset functionality
- User registration with new rules

Why This Approach:

- Tests the main change (password requirements)
- Tests related areas (login, registration)
- Doesn't test unrelated user features
- Efficient use of testing time
- Covers the risk areas

Example 2: User Details New Feature

My Real Example:

"If something new in user details, I will check that new thing in user details but will not test all about user management what is not in area of that new functionality."

✓ What I Test:

- The new user details feature
- User details page functionality
- Saving/updating user details
- User details validation
- Related user profile features

✗ What I Don't Test:

- All user management functions
- User permissions system
- User roles and groups
- Unrelated user features
- Entire user workflow

Example 3: When Devs Ask for Specific Testing

My Real Example:

"What is important in app I will test for security or devs tell me 'please check this when you testing that story.'"

When I Expand Testing:

- **Security-related changes:** Test broader security implications
- **Developer requests:** When dev specifically asks to check something
- **Critical app features:** Test important functionality more thoroughly
- **Integration points:** When changes affect multiple systems

1 Check the Change

Look at what was fixed or added - understand the scope of the change

2 Test the Fix/Feature

Verify the bug is fixed or the new feature works as expected

3 Test Related Areas

Test functionality in the same area or that connects to the change

4 Check Critical Functions (if needed)

Test important app functions or when devs specifically ask me to check something

Types of Regression Testing

Complete Regression Testing

Testing the entire application from scratch when major changes are made.

When to Use:

- Major system updates
- Architecture changes
- Multiple bug fixes
- Before major releases

Characteristics:

- Time-consuming
- Resource intensive
- Comprehensive coverage
- High confidence level

My Approach:

I rarely do complete regression testing because it takes too much time. I prefer focused testing unless it's a major release or devs specifically ask for it.

Partial Regression Testing (My Preferred Approach)

Testing only the affected modules and their related functionalities.

When I Use This:

- Bug fixes (most common)

Why I Prefer This:

- Faster execution

- Small feature additions
- Localized changes
- Most of my daily testing
- Time-efficient
- Focused and practical
- Covers the real risks

My Experience:

This is what I do most of the time. I test the changed area and related functionality. It's practical and efficient - if I tested everything for every change, I'd never finish!

Selective Regression Testing

Testing selected test cases based on code changes and impact analysis.

My Selection Criteria:

High Priority:

- The actual bug fix/feature
- Security-related areas
- What devs ask me to check

Medium Priority:

- Related functionalities
- Same module/page
- Connected workflows

Low Priority:

- Unrelated features
- Stable areas
- Different modules

Real-World Example: E-commerce Website

Scenario:

A bug was fixed in the payment processing module where credit card validation was failing for certain card types.

Regression Test Areas:

Direct Impact:

- Payment processing with all card types
- Credit card validation logic
- Payment confirmation flow
- Error handling for invalid cards

Indirect Impact:

- Order completion process
- Shopping cart functionality
- User account updates
- Email notifications

Test Cases to Execute:

- Verify payment with Visa, MasterCard, American Express
- Test payment with invalid card numbers
- Verify order completion after successful payment

- Test shopping cart persistence during payment
- Verify email confirmations are sent

Regression Testing Best Practices

✓ Best Practices:

- Focus on changed areas and related functionality
- Be efficient with testing time
- Test what devs specifically ask you to check
- Prioritize security-related changes
- Use TestRail test cases when available
- Document test results thoroughly
- Consider automation for repetitive tests

✗ Common Pitfalls:

- Testing everything without prioritization
- Spending too much time on unrelated areas
- Ignoring what developers specifically mention
- Not testing the actual bug fix thoroughly
- Missing related functionality testing
- Not considering security implications
- Poor time management during regression

My Philosophy: Regression testing should be practical and efficient. Focus on what changed and what's related. If you test everything all the time, you'll need too much time and won't be efficient with your testing effort.

Smoke Testing

Basic functionality verification

What is Smoke Testing?

Smoke testing is a preliminary testing approach that verifies the basic functionality of an application to ensure it's stable enough for further detailed testing. It's also known as "Build Verification Testing."

Key Characteristics:

- Quick and shallow testing

Purpose:

- Verify application launches successfully

- Tests critical functionalities only
- Performed after new build deployment
- Determines if build is stable for testing
- Usually automated
- Takes 30 minutes to 2 hours

- Check critical paths work
- Ensure basic functionality is intact
- Save time by catching major issues early
- Decide if detailed testing should proceed

My Smoke Testing Approach

My Experience:

I do smoke testing after new deployment on production. I check if everything works on production and that new features haven't crashed the production environment.

When I Do Smoke Testing

- **After production deployment:** When new code is deployed to production
- **Production verification:** Check that production environment works
- **New feature safety:** Ensure new features didn't break existing functionality
- **Critical check:** Verify the most important features still work

My Testing Approach

- **Manual testing:** For now I do manual testing
- **TestRail test cases:** I have test cases for smoke testing
- **Happy path focus:** Happy path testing with smoke testing
- **Most important tests:** Smoke tests are most important test cases from new feature

My Focus: After deploying new code to production, I do smoke testing to make sure everything works and new features haven't crashed the production environment.

My Production Smoke Testing Process

- 1 New Code Deployed to Production**
Development team deploys new features or bug fixes to production environment
- 2 Run Smoke Tests on Production**
Execute smoke test cases to verify production environment is working correctly

3 Check Critical Functionality

Test the most important test cases from new features and existing critical functions

4 Verify Production Stability

Confirm that new changes haven't crashed production and everything works as expected

What I Include in My Smoke Tests

My Approach:

Smoke tests are the most important test cases from new features plus happy path testing to make sure basic functionality works on production.

Most Important Test Cases:

- **New feature core functionality:** Main workflow of new features
- **Critical business functions:** Login, main navigation, core features
- **Happy path scenarios:** Successful user workflows
- **Integration points:** Areas where new features connect to existing system

Production Verification:

- **Application loads:** Site/app starts correctly
- **Authentication works:** Users can log in
- **Main features functional:** Core functionality not broken
- **No obvious crashes:** System stability on production

My TestRail Smoke Test Examples

Production Smoke Test Cases:

ID	Test Case Title (Happy Path Focus)
----	------------------------------------

S001	Verify that user can log in to the application on production
------	--

S002	Verify that main dashboard loads correctly on production
------	--

S003	Verify that navigation between main sections works
------	--

S004 Verify that new feature core functionality works correctly

S005 Verify that critical business functions are not affected by deployment

My Focus: These are the most critical test cases that verify production stability and that new features work without breaking existing functionality.

My Testing Approach: Manual vs Automation

Current Approach - Manual Testing

My Experience:

"For now I do manual testing" - I run through my TestRail smoke test cases manually on production.

- **Manual execution:** Go through TestRail test cases manually
- **Production testing:** Test directly on production environment
- **Real user experience:** See exactly what users would see
- **Flexible approach:** Can adapt tests based on what I observe

Future Automation Potential

- **Cypress automation:** Could automate smoke tests with Cypress
- **Faster execution:** Automated smoke tests run quicker
- **Immediate feedback:** Can run automatically after deployment
- **Consistent testing:** Same tests run every time

Current Reality: Manual testing works well for now and gives me direct control over production verification.

General Smoke Testing Process

Step 1: Build Deployment

New build is deployed to the test environment

Activities:

- Deploy latest build to test environment
- Verify deployment was successful
- Check application starts without errors

- Confirm environment setup is correct

Step 2: Execute Smoke Tests

Run predefined smoke test cases covering critical functionality

Test Areas:

- Application login/authentication
- Main navigation and menus
- Core business functions
- Database connectivity
- API endpoints (if applicable)
- File upload/download
- Search functionality
- Basic CRUD operations

Step 3: Analyze Results

Evaluate test results and make go/no-go decision

✓ PASS

All critical functions work.
Production is stable.

✗ FAIL

Critical issues found. Need
immediate fix.

⚠ CONDITIONAL

Minor issues found. Monitor
closely.

Smoke vs Sanity vs Regression Testing

Aspect	Smoke Testing	Sanity Testing	Regression Testing
Purpose	Verify build stability	Verify specific functionality	Verify existing features work
Scope	Broad but shallow	Narrow but deep	Broad and deep
When Performed	After new build	After minor changes	After any changes
Time Required	30 min - 2 hours	1-3 hours	Several hours to days
Automation	Usually automated	Can be manual or automated	Preferably automated

My Experience:

I focus on smoke testing after production deployments to ensure new features haven't broken the production environment. It's my first line of defense for production stability.

Smoke Testing Best Practices

✓ Do's:

- Keep test cases simple and focused
- Include most important test cases from new features
- Focus on happy path testing
- Test on production after deployment
- Verify new features don't crash production
- Document clear pass/fail criteria
- Update smoke tests with new features

✗ Don'ts:

- Don't include detailed test scenarios
- Don't test edge cases or negative scenarios
- Don't spend too much time on smoke testing
- Don't ignore smoke test failures
- Don't make smoke tests too complex
- Don't skip smoke testing after production deployment
- Don't test unrelated functionality

My Philosophy: Smoke testing after production deployment is critical. I check the most important functionality to ensure new features work and haven't broken existing features. It's about production stability and user confidence.

My Real-World Scenario: After Production Deployment

Situation:

New user profile feature was deployed to production. I need to verify that production is stable and the new feature works without breaking existing functionality.

My Smoke Test Process:

Happy Path Tests:

- User can log in to production
- Dashboard loads correctly
- Navigation works
- New profile feature works
- User can save profile changes

Critical Checks:

- No obvious crashes
- Main features still functional
- New feature integration works
- Production environment stable
- Users can complete main workflows

Result:

✓ **PASS:** All smoke tests passed. Production is stable, new profile feature works correctly, and existing functionality is not affected. Users can safely use the application.

Cypress Automation Testing

Modern end-to-end testing framework for web applications

My Cypress Experience

My Experience:

I've been using Cypress for about 1.5 years. I do more automation testing now and less manual testing. It's cool to write Cypress tests! I had been a programmer so I know how to write code and use global attributes that connect HTML and Cypress code.

What I Automate

- **All app functionality:** I test all functionality in the app
- **API calls:** Test a lot of API calls and intercept API calls
- **User management:** Creating users, user workflows
- **Complete workflows:** End-to-end user scenarios
- **Forms and navigation:** Complex form interactions

My Approach

- **Current project:** Writing Cypress tests for current project
- **Time allocation:** Few weeks to complete all tests for one role
- **Custom commands:** I use custom commands for reusable functionality
- **Local testing:** Run locally to see if tests work
- **Self-taught:** No one in company to help, learned independently

Programming Background Advantage: Having programming experience helps me write better Cypress tests and understand how to connect HTML elements with Cypress code using global attributes.

My Real Cypress Code Examples

<> My Custom Command: createTestUser

My Real Code:

This is my custom command for creating test users via API calls. I use this in multiple tests.

```
// In commands.ts

Cypress.Commands.add('createTestUser', (options: { firstName?:
string; email?: string } = {}) => {
  let testUserName = options.firstName || `Test User ${1752559752964
}`;
  const email = options.email || `testuser${1752559752964
}@example.com`;

  return cy.window().then((win) => {
    const tspData = JSON.parse(win.localStorage.getItem('TSP_DATA'));
    const authToken = tspData.jwtToken;

    return cy.request({
      method: 'POST',
      url: '/api/user/create',
      headers: { Authorization: `Bearer ${token}` },
      body: { email, firstName: testUserName, roleId: 1 }
    }).then((response) => {
      expect(response.status).to.eq(200);
      return { id: response.body.response.id, name: testUserName };
    });
  });
});
```

My Real Test: Case Creation Workflow

My Real Test:

This test verifies the complete case creation workflow with API interception and form interactions.

```
// My actual test for case creation

describe('New Case CTA buttons functionality', () => {
  beforeEach(() => {
    cy.login('admin'); // Use real login
  });

  it('should navigate to Add Case page and make expected API
requests', () => {
    cy.intercept('POST', '/api/case/filter').as('getCasesTablePage');
    cy.visit('/#/alerts?filterRange=7&view=list');
  });
});
```

```

cy.checkSpinnerNotVisible();

cy.wait('@getCasesTablePage').then((intercept) => {
  expect(intercept.response.statusCode).to.eq(200);
  expect(intercept.response.body.success).to.be.true;
});

cy.getDataTest('case-create-button').click();
cy.url().should('include', '/create-case');

// Fill form fields
cy.getDataTest('case-create-case-title').type('Test Case Title');
cy.getDataTest('case-create-case-type-
select').contains('Alert').click();

cy.intercept('POST', '/api/case/create').as('createCaseRequest');
cy.getDataTest('create-case-button').click();

// Verify success and API response
cy.get('.notification__text').contains('Case successfully
created');
cy.wait('@createCaseRequest').then((intercept) => {
  expect(intercept.response.body.response.title).to.eq('Test Case
Title');
});
});
});

```

My Custom getDataTest Command

My Utility Command:

I use data-test attributes for reliable element selection instead of CSS selectors.

```

// My custom command for data-test selectors
Cypress.Commands.add('getDataTest', (dataTestSelector) => {
  return cy.get(`[data-test="${dataTestSelector}"]`);
});

```

My Cypress Testing Approach

What Makes My Tests Effective:

- **API Integration:** Heavy use of API calls and intercepting

My Testing Strategy:

- **Role-based testing:** Complete all tests for one role at a time

- **Real data:** Create actual test users and data
- **Custom commands:** Reusable functions for common actions
- **Data attributes:** Use data-test for reliable element selection
- **Full workflows:** Test complete user scenarios

- **Local execution:** Run tests locally to verify functionality
- **Self-directed learning:** No team support, learned independently
- **Programming background:** Leverage coding experience
- **Comprehensive coverage:** Test all app functionality

My Challenge & Growth: The biggest challenge was not having anyone in the company to help with Cypress. I had to learn everything independently, but my programming background helped me understand the concepts quickly.

What is Cypress?

Cypress is a next-generation front-end testing tool built for the modern web. It enables you to write, run, and debug tests directly in the browser with real-time reloads and time-travel debugging capabilities.

✔ Key Features:

- Real-time browser testing
- Automatic waiting and retries
- Time-travel debugging
- Network traffic control
- Screenshots and videos
- Easy setup and configuration

⚡ Advantages:

- Fast test execution
- Developer-friendly syntax
- Excellent debugging capabilities
- Built-in assertions
- No WebDriver needed
- Great documentation

💻 Use Cases:

- End-to-end testing
- Integration testing
- Unit testing
- API testing
- Visual regression testing
- Component testing

🧑 Why I Like Cypress:

"It's cool to write Cypress tests!" - The syntax is intuitive, and with my programming background, I can create powerful tests that handle complex scenarios like API calls, user creation, and full workflows.

<> Getting Started: Installation & Setup

Step 1: Install Cypress

```
# Install via npm
npm install cypress --save-dev

# Or install via yarn
yarn add cypress --dev
```

Step 2: Open Cypress Test Runner

```
# Open Cypress GUI
npx cypress open

# Run tests in headless mode
npx cypress run
```

Essential Cypress Commands

Navigation & Interaction

cy.visit(url)

Navigate to a specific URL

Example: `cy.visit('/#/alerts?filterRange=7')`

cy.get(selector)

Get DOM element(s) by selector

Example: `cy.get('[data-test="submit-btn"]')`

cy.intercept(method, url)

Intercept network requests

Example: `cy.intercept('POST', '/api/case/create')`

cy.request(options)

Make HTTP requests

Example: `cy.request('POST', '/api/user/create')`

Assertions & Custom Commands

cy.should('be.visible')

Assert element is visible

Example:
`cy.get('.notification').should('be.visible')`

cy.wait('@aliasName')

Wait for intercepted request

Example: `cy.wait('@createCaseRequest')`

cy.getDataTest(selector)

My custom command for data-test

Example: `cy.getDataTest('case-create-button')`

cy.createTestUser(options)

My custom command for user creation

Example: `cy.createTestUser({firstName: 'John'})`

✓ What Works for Me:

- Use data-test attributes for reliable element selection
- Create custom commands for reusable functionality
- Intercept API calls to verify backend integration
- Use real data creation via API calls
- Test complete user workflows, not just individual features
- Leverage programming background for complex logic
- Run tests locally first to verify functionality

✗ Challenges I've Faced:

- No team support - had to learn everything independently
- Setting up complex API interactions initially
- Managing test data and cleanup
- Debugging timing issues with dynamic content
- Not running in CI/CD yet (only local execution)
- Balancing test coverage with development time

My Success Formula: Programming background + custom commands + API integration + data-test attributes = Comprehensive test coverage that actually works and catches real issues!

► Learning Resources

Video Tutorial

Comprehensive Cypress Tutorial

Complete guide covering installation, basic commands, advanced features, and best practices.

[🔗 Watch Tutorial on YouTube](#)

Official Documentation

Cypress Official Docs

Comprehensive documentation with examples, API reference, and guides for all Cypress features.

[🔗 Visit Cypress Documentation](#)

My Learning Journey & Recommendations

My Experience:

"I didn't have anyone in the company to help me with Cypress - that was a challenge. I had to learn everything independently using documentation and tutorials."

1. Start with the official Cypress documentation to understand core concepts
2. Follow hands-on tutorials to build your first tests
3. Practice with your own project - that's where real learning happens
4. Learn custom commands early - they save a lot of time
5. Focus on API testing and intercepting - very powerful features

6. Use data-test attributes for reliable element selection
7. Don't be afraid to ask questions in Cypress community forums

My Future Cypress Plans

Current State:

- **Local execution:** Running tests locally to verify functionality
- **Manual runs:** I trigger test runs manually when needed
- **Complete coverage:** Working on all functionality for current role
- **Custom commands:** Building library of reusable functions

Future Goals:

- **CI/CD Integration:** Set up automated test runs in pipeline
- **Cross-browser testing:** Test on different browsers
- **Parallel execution:** Run tests faster with parallel runs
- **Test reporting:** Better reporting and dashboards
- **Team adoption:** Help team members learn Cypress

Next Steps: While I currently run Cypress tests locally, I'm working towards integrating them into CI/CD pipelines for automated execution. My goal is to have comprehensive test coverage that runs automatically with each deployment.

Why I Choose Cypress

Programming Background Advantage:

- **Familiar syntax:** JavaScript-based, easy to understand
- **Code reusability:** Can create complex custom commands
- **API integration:** Natural to work with REST APIs
- **Debugging skills:** Can troubleshoot test issues effectively
- **Logic implementation:** Can handle complex test scenarios

Practical Benefits:

- **Fast feedback:** Immediate test results during development
- **Real browser testing:** Tests exactly what users experience
- **Network control:** Can mock and intercept API calls
- **Visual debugging:** See exactly what went wrong
- **Less manual testing:** Automation reduces repetitive work

Bottom Line: Cypress fits perfectly with my programming background and allows me to create comprehensive automation that covers all app functionality. It's enjoyable to write and powerful enough to handle complex scenarios like user creation, API testing, and complete workflows.

JMeter Performance Testing

Practical guide to Apache JMeter for load and performance testing

My JMeter Learning Journey

Full transparency: I'm currently learning JMeter through online courses and YouTube tutorials, and haven't used it on real projects yet. As a QA tester with experience in Cypress, Bruno/Postman, and manual testing, I recognize that performance testing is a crucial skill gap I want to fill.

Learning Resources I'm Using:

- [JMeter Tutorial Playlist](#) - Comprehensive video series
- [JMeter Complete Course](#) - 3+ hour detailed tutorial
- Official Apache JMeter documentation
- Practice with sample applications

Biggest Learning Challenge:

Understanding JMeter's component hierarchy and concepts like throughput, listeners, timers, and how they all work together was initially overwhelming. The terminology and relationships between components took time to grasp.

Future Goal: I'm building this knowledge base to strengthen my performance testing skills and hope to apply JMeter in real projects soon. If you're also learning JMeter, you're not alone - performance testing can seem complex at first, but the concepts become clearer with practice.

What is Apache JMeter?

Apache JMeter is a powerful open-source tool designed for load testing and performance measurement. It can test performance both on static and dynamic resources, web dynamic

applications, and simulate heavy loads on servers, groups of servers, networks, or objects to test their strength and analyze overall performance under different load types.

Key Features

- Multi-protocol support (HTTP, HTTPS, SOAP, REST)
- GUI and command-line modes
- Distributed testing capabilities
- Comprehensive reporting
- Extensible with plugins

Use Cases

- Load testing web applications
- API performance testing
- Database stress testing
- Functional testing
- Regression testing

Benefits

- Free and open source
- User-friendly GUI
- Cross-platform compatibility
- Active community support
- Detailed result analysis

▶ Getting Started with JMeter

1 Download & Install

Download Apache JMeter from official website

Steps:

- Download JMeter from <https://jmeter.apache.org/>
- Extract to desired directory
- Navigate to /bin folder
- Run jmeter.bat (Windows) or jmeter.sh (Linux/Mac)

2 Create Test Plan

Set up your first performance test

Steps:

- Right-click Test Plan → Add → Threads → Thread Group
- Configure number of threads (users)

- Set ramp-up period and duration
- Add HTTP Request sampler

3 Configure Requests

Define what endpoints to test

Steps:

- Right-click Thread Group → Add → Sampler → HTTP Request
- Enter server name/IP
- Set HTTP method (GET, POST, etc.)
- Add path and parameters

4 Add Listeners

View and analyze results

Steps:

- Right-click Thread Group → Add → Listener → View Results Tree
- Add Summary Report for metrics
- Add Graph Results for visual analysis
- Configure result file saving

Key JMeter Components

Thread Group

The foundation of any JMeter test plan. Controls how many virtual users will be simulated and how they ramp up over time.

Purpose:

HTTP Request

Creates HTTP requests to your application endpoints. You can test REST APIs, web pages, and any HTTP-based service.

Purpose:

Define user load patterns

Key Settings:

- Number of threads (users)
- Ramp-up period
- Loop count or duration

Define API endpoints to test

Key Settings:

- Server name/IP
- HTTP method (GET/POST/PUT)
- Path and parameters

Listeners

Display test results in various formats. Essential for analyzing performance metrics and identifying issues.

Purpose:

Collect and view results

Key Settings:

- View Results Tree
- Summary Report
- Graph Results

Assertions

Verify that responses meet expected criteria. Critical for ensuring your tests actually validate functionality.

Purpose:

Validate responses

Key Settings:

- Response code checks
- Response text validation
- Duration assertions

Real-World Test Scenarios

E-commerce Load Test

Testing online store during peak shopping hours

Configuration:

400 threads, 60s ramp-up, 30 minutes duration

Test Endpoints:

- Login
- Browse Products
- Add to Cart
- Checkout
- Payment

Expected Results:

< 3s response time, < 1% error rate, 100+ TPS

API Stress Test

Finding breaking point of REST API

Configuration:

Start: 50 users,
increase by 50 every 2
minutes until failure

Test Endpoints:

- User Registration
- Authentication
- Data Retrieval
- File Upload

Expected Results:

Identify max concurrent
users, monitor
memory/CPU usage

Database Performance Test

Testing database under heavy read/write load

Configuration:

200 threads, 120s
ramp-up, 20 minutes
duration

Test Endpoints:

- SELECT queries
- INSERT operations
- UPDATE statements
- Complex JOINS

Expected Results:

< 100ms query
response, no
connection timeouts

Practical JMeter Test Scenarios

Load Testing: E-commerce Website Peak Traffic

Configuration:

threads: 100 users
rampUp: 60 seconds
duration: 10 minutes
loops: Infinite

Endpoints Tested:

- Homepage load
- Product search
- Add to cart
- Checkout process

Success Criteria:

- Average response time < 3s
- 95th percentile < 5s
- Error rate < 1%
- Throughput > 50 req/sec

Stress Testing: API Breaking Point

Configuration:

threads: Start: 50, Max:
500 users
rampUp: 10 users every
30 seconds
duration: 30 minutes

Endpoints Tested:

- User authentication
- Data retrieval APIs
- File upload endpoints

Success Criteria:

- Find breaking point
- Monitor CPU/Memory usage
- Track error rates
- Recovery time after load

JMeter Best Practices

✔ Do's - Best Practices

- ✔ Start with small thread counts and gradually increase
- ✔ Use realistic ramp-up periods (not all users at once)
- ✔ Add think time between requests (1-3 seconds)
- ✔ Monitor server resources during tests
- ✔ Use CSV files for test data variation
- ✔ Run tests from multiple machines for high load
- ✔ Save results to files for analysis
- ✔ Use non-GUI mode for actual load testing

⚠ Don'ts - Common Pitfalls

- ⚠ Don't run performance tests on production
- ⚠ Don't ignore server-side monitoring
- ⚠ Don't use GUI mode for actual load testing
- ⚠ Don't forget to clear listeners for high-load tests
- ⚠ Don't test without baseline measurements
- ⚠ Don't run tests without proper test data
- ⚠ Don't skip validating test environment setup
- ⚠ Don't run long tests without incremental checkpoints

📄 Command Line Usage

For serious load testing, always use JMeter in non-GUI mode. The GUI is only for creating and debugging test plans. Command line mode provides better performance and resource utilization.

```
# Basic command to run test plan
jmeter -n -t TestPlan.jmx -l results.jtl
```

```
# Run with HTML dashboard report
jmeter -n -t TestPlan.jmx -l results.jtl -e -o /report/folder
```

```
# Override properties
jmeter -n -t TestPlan.jmx -l results.jtl -Jthreads=100 -Jrampup=300

# Distributed testing
jmeter -n -t TestPlan.jmx -R server1,server2,server3 -l results.jtl
```

Key Performance Metrics to Monitor

 **Response Time Metrics**

Average: **< 3 seconds**

90th Percentile: **< 5 seconds**

95th Percentile: **< 8 seconds**

 **Throughput Metrics**

Requests/sec: **> 100 req/s**

Transactions/sec: **> 50 TPS**

Concurrent Users: **1000+**

Error Metrics

Error Rate: **< 1%**

Timeout Rate: **< 0.5%**

Server Errors: **< 0.1%**

Pro Tip: Always establish baseline performance metrics before making changes. Run tests multiple times to account for variability, and monitor server resources (CPU, memory, disk I/O) alongside JMeter metrics for complete performance analysis.

Test Management Tools

JIRA, TestRail, and Kanban workflows for effective test management

My Experience:

I work as a QA Tester using tools like **TestRail** for test management, **Cypress** for automation, and **Bruno/Postman** for API testing. My daily work includes writing test cases, finding bugs, and collaborating with developers using **Jira Kanban**.

Key Achievement: Used JIRA and TestRail for 2 years across two major projects - network security threat detection and service provider platform.

What is Test Management?

Test management involves planning, organizing, and controlling testing activities throughout the software development lifecycle. **JIRA excels at bug tracking, user stories, and project management**, while **TestRail specializes in test case organization and execution tracking**. Together, they provide comprehensive coverage for quality assurance processes.

Key Activities

- Test planning and strategy
- Test case creation and organization
- Test execution tracking
- Defect management and reporting
- Progress monitoring and metrics

Benefits

- Improved test coverage and quality
- Better visibility into testing progress
- Efficient resource allocation
- Enhanced team collaboration
- Faster defect resolution

Test Management Tools Comparison

JIRA

Issue Tracking & Project Management

Strengths:

- ✓ Excellent for bug tracking and issue management
- ✓ Kanban board visualization
- ✓ Powerful workflow customization
- ✓ Great integration with development tools
- ✓ Comprehensive reporting and dashboards

Weaknesses:

- ⚠ Not specifically designed for test management
- ⚠ Limited test case organization
- ⚠ No built-in test execution tracking
- ⚠ Complex setup for testing workflows

Best For:

TestRail

Dedicated Test Management

Strengths:

- ✓ Purpose-built for test management
- ✓ Excellent test case organization
- ✓ Detailed test execution tracking
- ✓ Comprehensive test reporting
- ✓ Easy milestone and release management

Weaknesses:

- ⚠ Additional cost on top of JIRA
- ⚠ Requires integration setup
- ⚠ Learning curve for new users
- ⚠ Limited project management features

Best For:

- Test case management and organization
- Test execution and results tracking

- Bug tracking and defect management
- Agile project management
- Sprint planning and tracking
- Integration with development workflow

Pricing: Starts at \$7.50/user/month

- Test coverage analysis
- Regulatory compliance testing

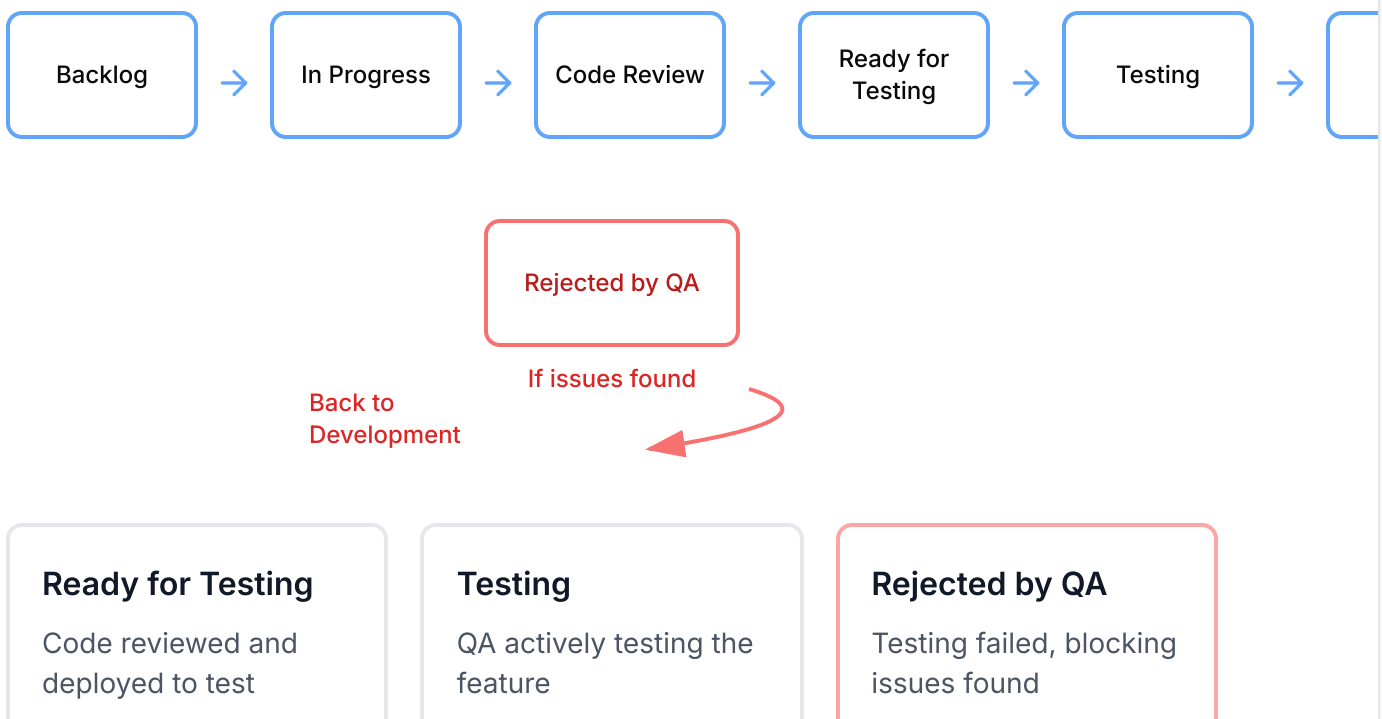
Pricing: Starts at \$37/user/month

|| JIRA Kanban Workflow for QA

My Experience:

I monitor JIRA "Ready for QA" column daily, read user stories and acceptance criteria, create test cases in TestRail, then move stories to UAT (pass) or "Disapproved by QA" (fail). Biggest challenge: developers not updating column status and empty user stories without acceptance criteria.

Kanban boards provide excellent visibility into work progress and help QA teams manage testing activities efficiently. When QA rejects a feature, it flows back to development for fixes, creating an iterative cycle until quality standards are met.



environment

Entry Criteria:

Deployed and ready for QA testing

QA Activities:

- Execute test cases
- Report defects

Entry Criteria:

QA assigned and testing in progress

QA Activities:

- Active testing
- Bug verification
- Regression testing

Entry Criteria:

Critical bugs or acceptance criteria not met

QA Activities:

- Document rejection reasons
- Create detailed bug reports
- Collaborate with dev team

JIRA Testing Workflow with Real Examples

1 User Story Creation

Product owner creates user stories with acceptance criteria

JIRA Elements:

Epic User Story Acceptance Criteria

Story Points

Testing Role:

Review requirements and identify testable scenarios

Real Example:

Login Feature User Story

"As a user, I want to reset my password so that I can regain access to my account"

Acceptance Criteria:

- Password reset link sent to registered email
- Link expires after 24 hours
- New password must meet security requirements

2 Sprint Planning

Team estimates and commits to sprint backlog

JIRA Elements:

Sprint Backlog Estimation Capacity Planning

Testing Role:

Estimate testing effort and plan test approach

3 Development

Developers work on tasks and update progress

JIRA Elements:

Tasks Subtasks Progress Tracking Time Logging

Testing Role:

Prepare test cases and test data

4 Testing

QA executes tests and reports defects

JIRA Elements:

Bug Reports Test Execution
Status Updates

Testing Role:

Execute tests, report bugs, verify fixes

Real Example:

Bug Report Example

"Login button becomes unresponsive after 3 failed attempts"

Steps to Reproduce:

1. Navigate to login page
2. Enter incorrect credentials 3 times
3. Observe login button behavior

5 Review & Deployment

Code review, testing sign-off, and deployment

JIRA Elements:

Code Review Testing Sign-off Deployment

Testing Role:

Final testing approval and deployment verification

TestRail Features & Capabilities

My Experience:

I use TestRail mostly separately from JIRA, though one project had TestRail extension in JIRA. I write brief, clear test case titles that are understandable for me and others. I organize test cases by features and link them to user stories.

Test Case Management

Organize and structure test cases efficiently

Capabilities:

- ✓ Hierarchical test case organization
- ✓ Test case templates and custom fields
- ✓ Test case versioning and history
- ✓ Shared test steps and reusable components

Example Test Case:

Password Reset Test Case

Preconditions: User account exists with valid email

Test Steps:

1. Navigate to login page
2. Click 'Forgot Password' link
3. Enter registered email address
4. Click 'Send Reset Link' button

Expected Results:

- Password reset form displays
- Email field accepts input
- Success message appears
- Reset email received within 5 minutes

Test Execution

Track test execution progress and results

Capabilities:

- ✓ Test run creation and management
- ✓ Real-time execution tracking
- ✓ Pass/fail/blocked status tracking
- ✓ Test result comments and attachments

Reporting & Analytics

Comprehensive test metrics and insights

Capabilities:

Integration

Connect with other tools in your workflow

Capabilities:

- ✓ Test coverage reports
- ✓ Progress and trend analysis
- ✓ Custom dashboards
- ✓ Executive summary reports

- ✓ JIRA integration for defect tracking
- ✓ CI/CD pipeline integration
- ✓ Automation tool integration
- ✓ API for custom integrations

JIRA + TestRail Integration

Many organizations use **JIRA for project management and bug tracking** and **TestRail for test case management and execution**, creating a powerful combination. The integration allows seamless workflow between project tracking and test execution.

Integration Benefits

- ✓ Automatic defect creation in JIRA from TestRail
- ✓ Bidirectional status updates
- ✓ Traceability between requirements and tests
- ✓ Unified reporting across both tools

Common Workflow

- 1 User story created in JIRA
- 2 Test cases created in TestRail
- 3 Test execution tracked in TestRail
- 4 Defects automatically created in JIRA
- 5 Bug fixes tracked in JIRA
- 6 Test results updated in TestRail

Best Practices

JIRA Best Practices

- ✓ Use clear and descriptive issue titles
- ✓ Always include steps to

TestRail Best Practices

- ✓ Organize tests in logical suites and sections
- ✓ Use consistent naming conventions

Integration Best Practices

- ✓ Set up automated defect creation from TestRail to JIRA
- ✓ Use consistent naming between

reproduce for bugs

- ☑ Add appropriate labels and components
- ☑ Link related issues (blocks, relates to)
- ☑ Keep status updated regularly
- ☑ Use proper priority and severity levels
- ☑ Include screenshots and logs when relevant

☑ Write clear and detailed test steps

- ☑ Include expected results for each step
- ☑ Use test case templates for consistency
- ☑ Regular test case reviews and updates
- ☑ Link test cases to requirements

tools

- ☑ Maintain traceability between requirements and tests
- ☑ Automate status updates where possible
- ☑ Regular sync between tool data
- ☑ Train team on both tools
- ☑ Establish clear workflow processes

Team Roles & Responsibilities

JIRA Responsibilities

Product Owner:

- Creates user stories with acceptance criteria
- Prioritizes backlog items
- Reviews and approves completed work

Developer:

- Updates task progress and status
- Logs time spent on development
- Fixes bugs reported by QA

QA Engineer:

- Reports bugs with detailed reproduction steps
- Verifies bug fixes
- Updates testing status on tickets

TestRail Responsibilities

QA Engineer:

- Creates and maintains test cases
- Executes test runs and records results
- Updates test case status (Pass/Fail/Blocked)

Test Manager:

- Reviews test coverage metrics
- Generates progress reports
- Manages test milestones and releases

Project Manager:

- Reviews testing progress dashboards
- Tracks overall quality metrics
- Makes go/no-go decisions based on test results

Key Metrics & KPIs to Track

JIRA Metrics

- | | |
|-----------------------|--------------------------|
| • Bug Detection Rate | Bugs found per sprint |
| • Bug Resolution Time | Average days to fix |
| • Sprint Velocity | Story points completed |
| • Defect Leakage | Bugs found in production |

TestRail Metrics

- | | |
|---------------------------|----------------------|
| • Test Execution Rate | Tests run vs planned |
| • Test Coverage | Requirements covered |
| • Pass/Fail Ratio | Success percentage |
| • Test Case Effectiveness | Bugs found per test |

Success Indicators:

85%+

Test Coverage

<2 days

Bug Resolution

<5%

Defect Leakage

❗ Common Integration Challenges & Solutions

Data Sync Issues

Common Problems:

- Status updates not syncing between tools
- Duplicate tickets being created
- Inconsistent data formats

Solutions:

- Set up automated sync schedules
- Use unique identifiers for linking
- Regular data validation checks

Permission Management

Common Problems:

- Different user roles between tools
- Access control conflicts
- Inconsistent permission levels

Solutions:

- Map roles consistently across tools
- Use single sign-on (SSO) when possible
- Document permission matrices

Training & Adoption

Common Problems:

- Team resistance to using both tools
- Inconsistent workflow adoption
- Knowledge gaps in tool features

Solutions:

- Provide comprehensive training sessions
- Create quick reference guides
- Designate tool champions in each team

Tool Selection Guide

Choose JIRA Only If:

- Small team with simple testing needs

Choose JIRA + TestRail If:

- Limited budget for additional tools
- Agile development with basic test tracking
- Focus on issue tracking over test management

- Large team with complex testing requirements
- Need detailed test case management
- Regulatory compliance requirements
- Comprehensive test reporting needed

Pro Tip: Start with JIRA for project management and basic bug tracking. As your testing needs grow and become more complex, consider adding TestRail for dedicated test management. The integration between both tools provides the best of both worlds.